FINAL PROJECT REPORT

Dog Behaviour and Human–Dog Interaction Recognition

Skeleton-Based Recognition from Indoor Home-Camera Video

COURSE	Pattern Recognition Systems Practice Module
GROUP	Group 28
TEAM MEMBERS	Zhao Jiayi · A0329873M Sun Wenjing · A0326610N
SUBMISSION DATE	28 October 2026

Table of Contents

Abstract 4
1. Introduction 4
1.1 Background and Motivation 4
1.2 Problem Statement 4
1.3 Objectives and Scope 5
1.4 Report Organisation 5
2. Datasets 6
2.1 Data Sources 6
2.2 Dog Behaviour Clips 6
2.3 Human-Dog Interaction Clips 7
2.4 Dog Pose Data (Hand-Labelled Evaluation Set and APT-36K) 8
2.5 Annotation and Data Preparation Pipeline 8
2.6 Data Splits and Leakage Control 9
3. Tools and Techniques 10
3.1 Software Stack and Frameworks 10
3.2 Object Detection (RTMDet) 11
3.3 Pose Estimation (RTMPose) 11
3.4 Multi-Target Tracking and Skeleton Extraction 12
3.5 Skeleton Preprocessing and Normalisation 14
3.6 Temporal Modelling (TCN) 15
3.7 Feature Engineering and Classical Classifiers 15
3.8 Ensemble Learning 15
3.9 Evaluation Methods 16
4. System Design 16
4.1 Overall System Architecture 16
4.2 Dual-Species Skeleton Extraction Pipeline 17
4.3 Repository and Module Structure 17
4.4 Frontend Prototype 18
5. Models 19
5.1 Model One: Dog Pose Estimation 19
5.1.1 Baseline (P0: RTMPose-m AP-10K) 19
5.1.2 Fine-Tuning on APT-36K (P1) 19
5.1.3 Further Adaptation (P2) 20
5.2 Model Two: Dog Behaviour Recognition 20

5.2.1 Motion-Aware TCN Backbone 21
5.2.2 Engineered Geometry Features 22
5.2.3 Geometry and Pairwise Fusion 24
5.2.4 Class-Specialist Models 24
5.3 Model Three: Human-Dog Interaction Recognition 25
5.3.1 Human-Dog Relation Features 25
5.3.2 ExtraTrees / Linear-SVM Soft-Vote Ensemble 27
5.3.3 Frozen CLIP Visual Branch 27
5.3.4 Probability-Level Fusion 28
5.4 Alternative Models Explored 28
6. System Performance 29
6.1 Experimental Setup 29
6.2 Evaluation Metrics 29
6.3 Dog Pose Estimation Results 29
6.4 Dog Behaviour Recognition Results 33
6.5 Human-Dog Interaction Recognition Results 34
6.6 Ablation Studies 35
6.7 Runtime and Deployment Considerations 36
7. Findings and Discussion 37
7.1 Key Findings 37
7.2 Error Analysis 37
7.3 Impact of Detection and Tracking Quality 37
7.4 Data Quality, Size and Leakage 38
7.5 Limitations 38
7.6 Future Work 38
8. Conclusion 38
References 39
Appendix 39
A. Installation and User Guide 39
B. Model Training and Reproduction Commands 40
C. Dataset and Model File Listing 42
D. Additional Results 43

Abstract

Indoor pet cameras can show an owner where a dog is, but not what it is doing or whether anyone has fed it or played with it. This project builds a CPU-only prototype that analyses short indoor clips and reports dog behaviour and human–dog interaction. The system has three models. Model One detects the person and the dog with RTMDet-m, tracks each with a bidirectional, score-aware box-selection rule and estimates 17 keypoints per subject with RTMPose-m. For the dog branch, the official AP-10K weights (P0) were compared with a version fine-tuned on the APT-36K dog subset (P1) on 36 hand-labelled indoor frames. P1 gained four AP points on APT-36K but was tied with P0 indoors (OKS AP 0.678 versus 0.693 with detected boxes; paired bootstrap interval of the mean OKS change −0.024 to +0.032), so P0 was kept. Model Two classifies five dog behaviours (walking/running, standing, sitting, lying/sleeping and eating/drinking) with a motion-aware temporal convolutional network fused with an 834-dimensional engineered geometry classifier and two pairwise specialists. On 137 project clips it reaches 77.37% out-of-fold (OOF) accuracy and 76.47% Macro-F1 under grouped five-fold cross-validation. Model Three classifies four human–dog interactions (no interaction, hand feeding, petting and playing together) with an ExtraTrees/linear-SVM ensemble over 352 human–dog relation features, fused with a frozen CLIP visual branch. On 97 development clips it reaches 77.32% OOF accuracy and 76.13% Macro-F1. All splits are grouped by dog, person and source video. The models are served through a FastAPI back end and a React mobile web application that shows a daily story, per-clip timelines and a weekly summary.

1. Introduction

1.1 Background and Motivation

Pet ownership has grown in recent years, and many owners regard their dogs as family members and want to know how they are while the owners are away from home. Pet cameras have become popular as a result. Grand View Research valued the global pet-monitoring camera market at about USD 198 million in 2024 and expects it to reach about USD 454 million by 2030, a compound annual growth rate of about 12.6%. The same report names the view of pets as family members, and owners' wish to check on their pets' safety and well-being while away, as major drivers of this growth.

Most current pet cameras answer only *where* the pet is. They detect a cat or dog, pan to follow it, or send an alert when they detect movement or barking. Owners want to know *what* the dog is doing: whether it is eating and resting normally, and whether family members or carers have fed it or played with it. Answering these questions today means reviewing recorded footage by hand.

A single frame is often not enough to answer them. Eating and sniffing with the head lowered look alike in one image, as do sleeping and ordinary lying, or petting and standing next to the dog. The differences show in how the motion develops over a few seconds and in how the person and the dog are positioned relative to each other. This project therefore moves from locating the pet towards recognising what the pet is doing and how it interacts with people.

1.2 Problem Statement

Given a short video from an indoor home camera that contains one principal dog and possibly one principal person, the system must:

1. classify the dog's behaviour into one of five classes: walking/running, standing, sitting, lying/sleeping or eating/drinking; and
2. when a person is present, classify the human–dog interaction into one of four classes: no interaction, hand feeding, petting or playing together.

This raises four difficulties:

- **Domain shift.** Public animal pose data contain few indoor home-camera scenes. Indoor dogs are often seen from above, are partly hidden by furniture, or lie curled up.
- **Small data.** Clips have to be cut and labelled by hand, so only a few hundred are available. Neighbouring clips from one video, or the same dog, must not appear on both sides of an evaluation split.
- **Subtle temporal cues.** Several classes differ mainly in duration or motion pattern, for example sustained head lowering for eating, or a hand that approaches, holds and withdraws for hand feeding.
- **Commodity hardware.** The system must run on an ordinary laptop CPU.

1.3 Objectives and Scope

The project has four objectives:

1. **Dog pose estimation (Model One).** Evaluate the AP-10K-pretrained RTMPose-m dog model on indoor frames, test whether fine-tuning on the APT-36K dog subset improves it, and select the weights used to extract skeletons.
2. **Dog behaviour recognition (Model Two).** Recognise the five behaviours from the temporal sequence of the dog skeleton and its detection box.
3. **Human–dog interaction recognition (Model Three).** Recognise the four interaction classes from synchronised human and dog skeletons, their spatial relations and auxiliary visual context.
4. **End-to-end prototype.** Provide reproducible training and evaluation procedures and a complete user workflow. The user uploads a clip, the system analyses it automatically, and the application shows the behaviour, the interaction, confidence scores and a timeline.

The scope is limited to indoor clips with one principal dog and at most one principal person. Clips with several dogs or people, and audio, are outside the scope of the current system. The labels describe observable actions only; they are not assessments of the dog's health or emotional state.

1.4 Report Organisation

The rest of the report is organised as follows:

- **Section 2** describes the datasets, the annotation pipeline and the grouped splits.
- **Section 3** introduces the tools and techniques: detection, pose estimation, tracking, preprocessing, temporal modelling, feature engineering, ensembles and evaluation.
- **Section 4** presents the system architecture, the repository structure and the front-end prototype.
- **Section 5** details the three models.
- **Section 6** reports their performance, ablations and runtime.
- **Section 7** discusses the findings, errors and limitations.
- **Section 8** concludes.

The appendices give installation instructions, reproduction commands, a file listing and additional results.

2. Datasets

2.1 Data Sources

The project combines public pose data with a project-curated video collection. For dog-pose adaptation, the dog subset of APT-36K is used while retaining its official training and validation separation. It provides animal pose supervision that is independent of the project videos and is converted to the 17-keypoint convention used by the dog-pose branch. The hand-labelled pose evaluation images are kept separate from behaviour and interaction labels and are described in Section 2.4.

The dog-behaviour and human–dog interaction datasets were assembled from project-contributed clips and publicly available research-use videos collected from sources including YouTube, Pexels, Mixkit, InterPet and SmartHomeBench. Selection prioritised indoor, third-person and relatively stable-camera footage in which the principal subjects were visible. A limited number of outdoor or partially occluded clips were retained when the action boundary remained unambiguous, providing moderate viewpoint and scene variation. Audio is not used by the current system.

Every original video is assigned a source_video_id, and each dog and person is represented by an anonymous dog_id or person_id. Source references and quality notes are retained in the manifests so that each short clip can be traced to its original video without storing personal names.

2.2 Dog Behaviour Clips

The dog-behaviour dataset contains **170 valid project clips** drawn from **63 source videos** of **61 anonymous dogs**. Of these, 96 clips come from Bilibili videos and 74 from YouTube videos, including SmartHomeBench footage. Each clip shows one principal dog with one stable behaviour. Clips range from 1.343 to 10.344 seconds, with a mean of 5.45 seconds and a total of 926.6 seconds.

The five behaviour classes are defined as follows:

- **Walking/running:** the dog moves its whole body across the floor with alternating paw steps.
- **Standing:** the dog is supported by all four extended legs without travelling. Small head movements and looking around are allowed.
- **Sitting:** the hindquarters rest on the floor while the front legs remain extended.
- **Lying/sleeping:** the trunk rests on the floor or furniture, either sleeping or awake.
- **Eating/drinking:** the dog's head is lowered into a bowl, a container or food on the floor, and it is eating or drinking.

When posture and activity conflict, the clip is labelled by posture (Section 2.5).

The manifest also contains 32 rows that are marked as invalid and excluded from every split:

- 26 clips of a dog playing alone, a class that is not part of the five-class model;
- four clips in which two dogs move in view;
- one clip whose label boundary overlaps another segment; and
- one night-time infrared clip whose skeletons were unusable (Section 3.4).

The clips are split by dog and source video (Section 2.6). The 33-clip test set contains 9 dogs, and the 137-clip development set contains the other 52 dogs. The development set is divided into five grouped folds of 29, 28, 26, 26 and 28 clips.

Walking/running and standing are under-represented in the project data. To compensate, 31 curated one-second APT-36K skeleton sequences (15 frames each) are added to the **training portion of every fold**. These sequences never appear in validation or testing.

Behaviour class	Development (5-fold OOF)	Test	External APT (training only)	Total
Walking / Running	22	6	28	56
Standing	28	8	3	39
Sitting	30	6	0	36
Lying / Sleeping	40	7	0	47
Eating / Drinking	17	6	0	23
Total	**137**	**33**	**31**	**201**

2.3 Human-Dog Interaction Clips

The interaction dataset contains **121 valid clips** drawn from **80 source videos**, with 69 anonymous dog identifiers and 69 anonymous person identifiers. Each clip contains one principal person and one principal dog and has one stable interaction label. The clips range from 1.679 to 10.239 seconds, with a mean duration of 4.830 seconds. Label transitions are cut into separate clips rather than assigning multiple sequential actions to one sample.

The four interaction classes are defined as follows:

- **No interaction:** the person and dog are both present, but there is no sustained physical contact, hand feeding or mutually engaged play. Speech, looking or gestures without contact remain in this class.
- **Hand feeding:** food or a reward is transferred from the person's hand, or from a hand-held container, towards the dog's mouth. Independent eating from a bowl is not labelled as human–dog interaction.
- **Petting:** the person's hand maintains contact with the dog's head or trunk and performs a small stroking movement. Sustained holding or embracing is also included, whereas incidental contact is excluded.
- **Playing together:** the person and dog are both actively engaged, for example through chasing, toy tugging or repeated large coordinated movements. A dog playing alone is not included in this class.

The final fixed split is shown below. The 24-clip test set is balanced with six clips per class; the remaining 97 clips form the development set used for grouped five-fold evaluation.

Interaction class	Train	Validation	Test	Total
No interaction	23	4	6	33
Hand feeding	18	5	6	29
Petting	19	6	6	31
Playing together	15	7	6	28
Total	**75**	**22**	**24**	**121**

2.4 Dog Pose Data (Hand-Labelled Evaluation Set and APT-36K)

Model One uses two pose datasets. Neither is used to train the behaviour or interaction classifiers.

APT-36K dog subset. The dog annotations of APT-36K provide the training data for the fine-tuned pose variant (Section 5.1.2). The subset contains 1,193 frames and 2,245 dog annotations from 27 source videos, all at 1920×1080 resolution. A total of 180 annotations contain no labelled keypoint and are removed, leaving 2,065 dog instances. The data are split by source video, so that frames from one video never appear in both partitions. APT-36K was chosen instead of AP-10K because the baseline model had already been trained on AP-10K. Its keypoints are either labelled or unlabelled; there is no separate state for a labelled but occluded joint.

APT-36K split	Source videos	Images	Dog instances
Training	24	983	1,691
Validation	3	209	374

Indoor evaluation set. Model selection is based on 36 frames taken from the project's indoor home-camera clips. The frames come from 28 dogs in 28 source videos and contain one dog each. They were selected so that each of the six behaviour labels in use at selection time (walking/running, standing, sitting, lying/sleeping, eating/drinking and playing alone) contributes six frames. Playing alone was later removed from Model Two because it could not be labelled consistently; this does not affect the pose evaluation, which never uses behaviour labels. The grouping is used only to break pose accuracy down by posture.

All 17 AP-10K keypoints were labelled by hand by one annotator. Only visible keypoints whose position could be determined were labelled; joints hidden by the body or furniture, or outside the image, were left unlabelled rather than guessed, and they are excluded from every score. This follows the AP-10K convention and allows a direct comparison with the official AP-10K results in Section 6.3. Severely blurred frames, in which a human annotator could not locate the joints, were not included.

Item	Value
Frames	36 (6 per behaviour label)
Dogs / source videos	28 / 28
Keypoint slots (36 × 17)	612
Labelled visible keypoints	547 (89%)
Unlabelled (occluded or undeterminable)	65 (11%)
Most frequently unlabelled	right knee and right back paw (10 each), left back paw (8), left knee (6)

None of these frames, and none of their source videos, are used to train any pose model.

2.5 Annotation and Data Preparation Pipeline

The dog-behaviour data are prepared in six steps (Figure 2-1), from online videos to the skeleton sequences consumed by Model Two. The interaction clips pass through the same skeleton-extraction steps.

1. **Video collection.** Videos were searched for on public platforms such as YouTube and Bilibili and in research collections such as SmartHomeBench. Footage resembling a home camera was preferred: a fixed, usually elevated indoor viewpoint, a stable camera and a clearly visible dog. Each original video receives a source_video_id and each dog an anonymous dog_id.

2. **Behaviour clipping.** Each video was watched and cut by hand into short clips in which the dog shows one stable behaviour. Most clips last 5–8 seconds (140 of the 170 valid project clips; median 5.0 s). When posture and activity conflict, the clip is labelled by posture; for example, a dog chewing a toy while lying down is labelled lying/sleeping. Start and end times are recorded in a manifest so that every clip can be re-cut from its source. The actual length of every clip file was checked with ffprobe against its labelled interval, and clips whose file ran more than one second longer than the label were re-cut from the original video.
3. **Grouped split.** Clips are assigned to partitions by dog and source video before any frame is sampled (Section 2.6).
4. **15 fps sampling.** Every clip is resampled to 15 frames per second by keeping the first frame of each 1/15-second interval. A common frame rate makes clips recorded at 24–60 fps comparable in time.
5. **Skeleton extraction with Model One.** RTMDet and the tracking rule select the dog (and, for interaction clips, the person) in each frame, and RTMPose estimates 17 keypoints inside the selected box (Sections 3.2–3.4). Annotated skeleton videos of a sample of clips were reviewed by eye, and the tracking rule was revised whenever a systematic failure was found.
6. **Skeleton sequences.** Each clip is stored as one $2 \times T \times 17 \times 3$ skeleton array with the selected boxes, which is handed to Model Two (dog stream only) and Model Three (both streams).

The hand-labelled pose evaluation set (Section 2.4) branches off after the split. Three to six candidate frames per clip were chosen automatically to cover different moments of the clip, ranked by sharpness, dog size and truncation, and then reduced by within-clip similarity. Dog boxes and the 17 keypoints were labelled in LabelMe, with model predictions used only as a starting point that the annotator corrected by hand for every frame. The final 36 evaluation frames were selected from these labelled frames.

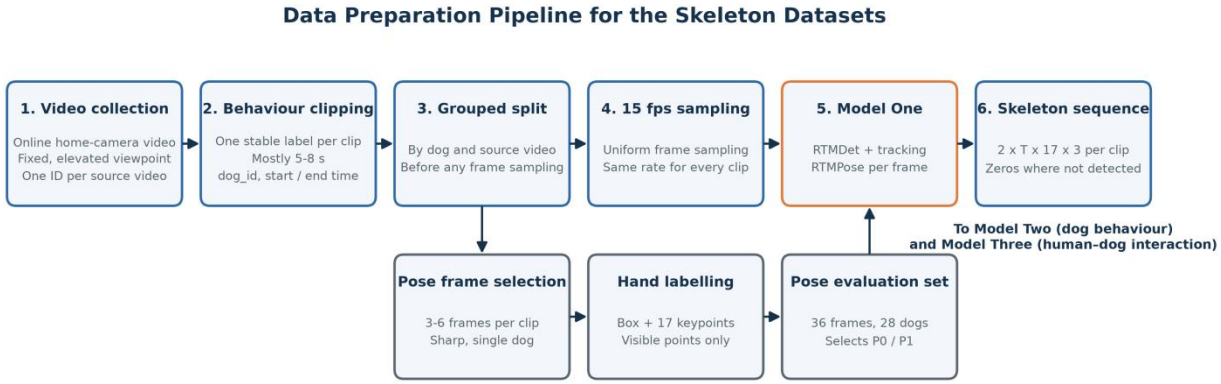


Figure 2-1. Data preparation pipeline. Online home-camera videos are cut by hand into single-behaviour clips, split by dog and source video, sampled at 15 fps and converted by Model One into two-stream skeleton sequences for Models Two and Three. The lower branch shows how frames from the same clips were selected and hand-labelled to form the pose evaluation set used to choose the dog pose weights.

2.6 Data Splits and Leakage Control

The datasets are split at subject and source-video level rather than by individual frames or randomly selected clips. All clips derived from the same source_video_id remain in one partition. Dog and person identifiers are also checked jointly: if clips are connected through a shared dog or person, the complete connected identity group is assigned to the same partition. This prevents visually similar neighbouring clips, repeated backgrounds and repeated subjects from appearing on both sides of an evaluation boundary.

For the interaction dataset, the fixed split contains 75 training, 22 validation and 24 test clips. There is no overlap in dog_id, person_id or source_video_id between these partitions. Model development combines the 75 training and 22 validation clips into a 97-clip development set. Stratified grouped five-fold cross-validation is then applied to 47 connected human–dog identity components. In each round, four folds train the model and the remaining fold produces predictions for clips that were not used to fit that fold. Concatenating the five validation predictions yields one out-of-fold prediction for every development clip.

The final interaction ensemble is fitted on all 97 development clips, while the 24-clip test set is excluded from weight fitting and reported only as supporting evidence. Video cutting, frame sampling, pose extraction, feature generation and augmentation inherit the previously assigned split; they are not performed before grouping. The same principle is used for dog-behaviour data, with dog_id and source_video_id acting as the primary grouping keys.

3. Tools and Techniques

3.1 Software Stack and Frameworks

The system is divided into a Python model and back-end stack and a JavaScript mobile web front end. All models run on the CPU, so the complete system can be used on an ordinary laptop without a GPU.

Layer	Tools	Role
Language and environment	Python 3.11 (Conda)	Inference, evaluation, feature extraction and the back end
Pose training	PyTorch 2.1, MMEngine, MMCV 2.1, MMDetection 3.2, MMPose 1.3	Fine-tuning RTMPose on APT-36K (P1) and exporting checkpoints
Inference	ONNX Runtime, OpenCV, NumPy	RTMDet and RTMPose inference, video decoding and 15 fps sampling
Recognition models	PyTorch, scikit-learn 1.9, joblib	Motion-aware TCN, ExtraTrees, linear SVM, PCA and logistic regression; saved ensembles
Visual features	Hugging Face Transformers	Frozen CLIP ViT-B/32 encoder for the interaction visual branch
Video tools	FFmpeg / ffprobe	Clip cutting, re-encoding and length checks
Annotation	LabelMe	Dog boxes and keypoints for the pose evaluation set
Back end	FastAPI, Uvicorn	REST API for uploading clips, tracking analysis jobs and returning results
Front end	React 18, React Router, Vite 5	Mobile web application
Testing	pytest, Vitest	Back-end and model tests; front-end summary-logic tests
Plots	Matplotlib	Report figures

Back end. The FastAPI service stores each uploaded video together with its recording time and camera name, and analyses it in a background worker. Analysis runs one clip at a time because CPU inference is heavy. Each analysis job reports four stages (detection, keypoints, behaviour and interaction) so that the client can show progress. The main endpoints upload a clip, query a job, list clips within a date range, and return one clip's result and cover image. Results are written as JSON files next to each clip, so no separate database is required.

Front end. The mobile web application is built with React and Vite. Its pages cover the daily story, uploading, analysis progress, the result of one clip and a weekly summary. All data access goes through one API module; a mock mode serves the same pages from sample data, so that the interface can be developed and demonstrated without running the models. During development, Vite forwards /api requests to the FastAPI service, and a phone on the same network can open the application directly.

The P1 fine-tuning used an earlier Python 3.10 environment with the same OpenMMLab versions. MMCV provides no Windows build for the CUDA 12.8 runtime required by the available RTX 5060 GPU, so training also ran on the CPU.

3.2 Object Detection (RTMDet)

RTMDet is a one-stage real-time object detector built on a CSPNeXt backbone. The project uses RTMDet-m with the official COCO weights and does not retrain it. Of the 80 COCO categories, only the person and dog outputs are retained.

A COCO-trained detector frequently assigns indoor dogs to another quadruped category. On the indoor evaluation set, the dog was localised correctly in every frame, but in 9 of the 36 frames another category scored higher than “dog” (eight as cat and one as cow). All quadruped categories (cat, dog, horse, sheep, cow, bear and similar) are therefore merged into a single dog class, and the merged box with the highest score is used. A box must score at least 0.3 to start a new target; lower-scoring boxes are accepted only to continue an existing track (Section 3.4).

With this post-processing, a dog was found in all 36 evaluation frames. Compared with the hand-drawn boxes, the detected dog boxes have a median IoU of 0.83 and a minimum of 0.63. The merging rule assumes that the dog is the only animal in view: a real cat in the same room would also be treated as a dog.

3.3 Pose Estimation (RTMPose)

RTMPose is a top-down pose estimator: the image region inside a detection box is cropped and resized, and the model predicts the keypoints of the single subject in that crop. RTMPose-m uses a CSPNeXt-m backbone and an RTMCC head. Instead of regressing a heatmap, it applies SimCC, which treats the x- and y-coordinate of every keypoint as two separate classification problems over sub-pixel bins (split ratio 2.0). This keeps the model light enough for CPU inference.

Two RTMPose-m models are used:

Branch	Training data	Input size	Keypoints
Human	COCO	256×192	COCO-17
Dog	AP-10K	256×256	AP-10K-17: eyes, nose, neck, tail root, and shoulder–elbow–front paw and hip–knee–back paw on each side

The confidence of a keypoint is the peak value of its SimCC distribution. It is not a probability and can exceed 1 (up to about 1.7) or fall slightly below 0. Confidence is therefore clipped to $[0.001,1]$, so that a value of 0 always means that the subject was not detected. Both models are exported to ONNX and run with ONNX Runtime on the CPU.

Dog pose accuracy is measured with Object Keypoint Similarity (OKS), the standard COCO and AP-10K metric. For one dog,

$$\mathrm{OKS} = \frac{\sum_i \exp\left(-\frac{d_i^2}{2s^2k_i^2}\right)\delta(v_i > 0)}{\sum_i \delta(v_i > 0)},$$

where d_i is the distance between the predicted and labelled position of keypoint i, s^2 is the object area (the area of the hand-drawn box, since no segmentation is available), k_i is the AP-10K per-keypoint constant, and $\delta(v_i > 0)$ restricts the sum to labelled keypoints. An OKS of 1 means perfect agreement. OKS AP is the COCO-style average precision over OKS thresholds from 0.50 to 0.95; AP50 and AP75 use single thresholds. The Percentage of Correct Keypoints, PCK@α, is the proportion of labelled keypoints whose error is no more than α times the long side of the hand-drawn box; $\alpha = 0.05$ and 0.1 are reported.

3.4 Multi-Target Tracking and Skeleton Extraction

Every clip is sampled at 15 fps. RTMDet is first run on the whole clip; the person and the dog are then tracked separately, and RTMPose is applied inside the selected box in each frame. The output of a clip is

$$\mathbf{S} \in \mathbb{R}^{2\times T\times 17\times 3},$$

where stream 0 is the person (COCO-17), stream 1 is the dog (AP-10K-17), T is the number of frames, and each joint stores its x- and y-coordinate in pixels and its confidence. A frame in which the target is not found is filled with zeros. The selected box and its detection score are stored alongside each frame. Dog-behaviour clips export only the dog stream: in an early batch export, 30 of 93 dog-only clips contained false "person" tracks on static objects such as photographs, coats and picture-in-picture overlays.

A frame can contain more than one candidate box, and a dog lying down often receives a low detection score. The box-selection rule must therefore keep the same subject across the clip without discarding low-confidence frames of that subject. The final rule is summarised below.

Rule	Condition	Purpose
Starting frame	The highest-scoring box in the clip with score ≥ 0.3; if none exists, the highest-scoring box with score ≥ 0.1; otherwise the target is absent from the clip	Start from the most reliable detection
Bidirectional tracking	Track forwards to the end and backwards to the start of the clip from the starting frame	Recover frames before the first confident detection
Normal continuation	Score ≥ 0.3 and IoU > 0 with the previous box; the box with the largest IoU is chosen	Follow the same subject
Low-score continuation	0.1 ≤ score < 0.3 and IoU ≥ 0.3 with the previous box	Keep lying, distant or partly occluded dogs
Track loss	No box accepted for 3 consecutive frames (0.2 s)	Allow a different box to take over
Recovery	The last position is remembered and the continuation rules keep being applied around it; after a loss, a box with score ≥ 0.3 elsewhere may also take over	Re-attach the subject after a short occlusion

The rule was developed in six versions, each motivated by failures found by visually reviewing extracted skeletons:

Version	Change	Problem addressed
1	Highest-scoring box in every frame	—
2	Highest-scoring box in the first frame, then the box overlapping the previous one most, falling back to the highest score	Version 1 switched between dogs in multi-dog frames

3	Overlap only, with no fallback to score	Version 2 could still jump to another animal
4	Low-score continuation and re-detection after 3 lost frames	Lying dogs scored 0.1–0.3 and were dropped
5	Whole-clip detection, bidirectional tracking, fallback start at the clip's best box, low-score IoU relaxed from 0.5 to 0.3	Frames before the first confident box were lost; some lying dogs never reached 0.3
6 (final)	Remember the last position after a track loss	Dogs returning from behind a person with a low score were not re-attached

The changes recovered most of the frames in which the detector had found the dog but with a low score. In a human–dog play clip, the dog was detected in 55 of 90 frames with version 3 and in 78 with version 4. A night-time clip of a curled-up sleeping dog rose from 78% to 100% of frames with bidirectional tracking, and a distant dog lying on its back rose from 0% (no box ever reached 0.3) to 78% with version 5 and 90% with version 6. In the first complete export, 12 clips had their target in fewer than 80% of frames; version 6 reduced this number to nine. The remaining cases are mostly genuine absences (the dog leaves the view or is fully hidden by a person), corrupted video frames, or boxes that cover only the dog's head. One night-time infrared clip, in which the detector scored the dog at only 0.04–0.3, produced unusable skeletons and was excluded from the dataset.

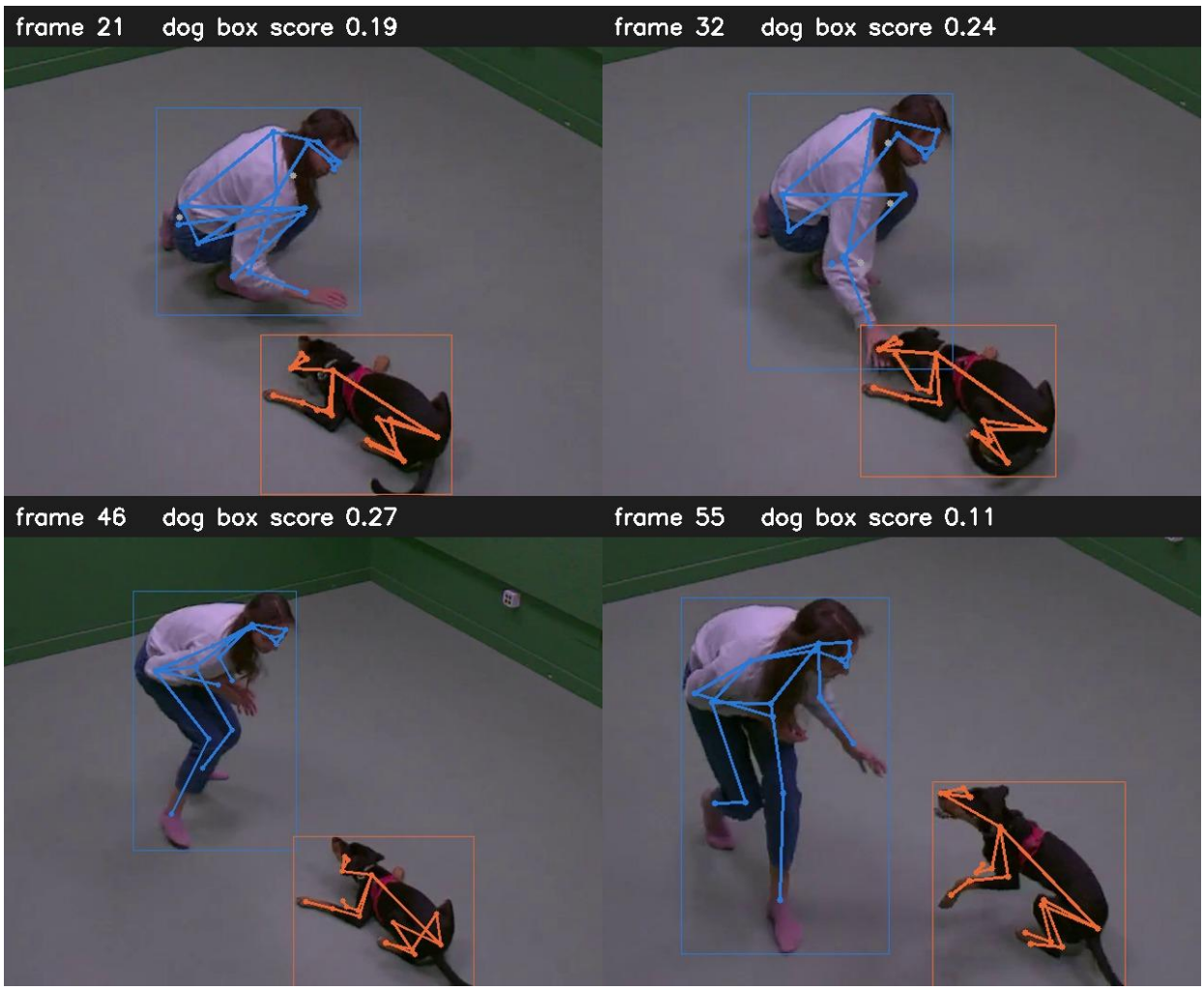


Figure 3-1. Four frames from a human–dog play clip in which the dog lies on the floor. The person (blue) and dog (orange) are shown with their boxes and skeletons; each title gives the dog-box detection score. All four dog boxes score between 0.11 and 0.27 and would be discarded by a fixed 0.3 threshold; the low-score continuation rule keeps them because they overlap the dog box of the previous frame.

Detection and pose estimation therefore remain the main source of missing data. Frames in which the detector produces no usable box cannot be recovered by any selection rule, and lying/sleeping is the weakest posture in both detection and pose accuracy (Section 6.3). The downstream models must therefore tolerate zero-filled frames, which they do through interpolation, frame masks and confidence-weighted pooling.

3.5 Skeleton Preprocessing and Normalisation

Raw skeletons are expressed in image pixels, contain zero-filled frames and vary in length, so they are preprocessed before entering the recognition models. All steps operate on the stored $2 \times T \times 17 \times 3$ arrays and are shared by training and inference.

Missing keypoints. A joint counts as observed in a frame when its confidence is above zero. For each joint, the x- and y-coordinates of unobserved frames are filled by linear interpolation over the frame index, using the nearest observed frames. Before the first observation and after the last one, the nearest observed value is held. Interpolated points keep a confidence of zero, so the downstream confidence weighting still treats them as unobserved. A joint that is never observed in a clip is left at zero.

Body-centred normalisation. Each frame is translated and scaled so that skeletons from near and distant, small and large subjects become comparable.

- **Dog.** The centre is the midpoint of the neck and the tail root when both are visible. Otherwise it is the mean of the visible joints. The scale is the larger of the horizontal and vertical extent of the visible joints.
- **Person.** The centre is the hip midpoint. When the hips are missing, the shoulder midpoint is used instead, and otherwise the mean of the available joints. The scale is defined in the same way as for the dog.

To avoid dividing by a near-zero extent when only a few joints are visible, the scale is bounded below by 2% of the longer image side. Every joint becomes $(\mathbf{p} - \mathbf{c})/s$, where $\mathbf{c}$ is the centre and s the scale. Unavailable joints and empty frames are set to zero.

Global motion. Body-centred normalisation removes the movement of the dog through the image, which is the main cue for walking/running. Six global-motion channels are therefore computed from the dog detection box:

- the two-dimensional displacement of the box centre from the first frame, normalised by the image size;
- the two-dimensional velocity of the box centre; and
- the logarithm of the box scale relative to the first frame, together with its velocity.

Missing box centres are interpolated over time and smoothed with an edge-padded moving average of up to five frames. When no box is available, the same quantities are computed from the keypoints instead.

Temporal resampling. Every clip is mapped to $T = 32$ frames using one set of indices shared by all streams, so that the person, the dog and the boxes stay synchronised.

- Clips with at least 32 frames are sampled at 32 evenly spaced positions, rounded to the nearest frame.
- Shorter clips keep all their frames and are padded with zeros at the end.

- A binary frame mask marks real frames (1) and padding (0), so that padded positions do not contribute to pooling.

During training, each sampling position is additionally jittered by up to ±0.35 of the sampling step. The first and last positions stay fixed, and the order of the positions is preserved.

Skeleton augmentation (training only). Each training sequence receives:

- a random rotation of ±10° and a scaling of 0.9–1.1, applied to both the normalised joints and the global-motion displacement and velocity;
- Gaussian coordinate noise with a standard deviation of 0.01; and
- with probability 0.3, the removal of one randomly chosen joint over the whole clip.

The resulting dog tensor has shape $3 \times 32 \times 17$ (coordinates and confidence, time, joints). It is accompanied by the six global-motion channels and the frame mask.

3.6 Temporal Modelling (TCN)

Temporal Convolutional Networks (TCNs) model a sequence using one-dimensional convolutions along the temporal axis. In comparison with a recurrent model, all time positions within a layer can be processed in parallel. Dilated convolution enlarges the temporal receptive field without reducing sequence length, while residual connections support the training of deeper temporal models. These properties make a TCN suitable for recognising actions from short skeleton sequences, where both frame-to-frame joint motion and longer-duration posture patterns are important.

For a residual temporal block, the output can be expressed as

$$\mathbf{H}_{l+1} = \mathrm{ReLU}(\mathbf{H}_l + F_l(\mathbf{H}_l)),$$

where F_l contains temporal convolutions, normalisation, activation and dropout. The dilations used in this project increase across successive blocks, allowing the model to combine local motion with information from a wider temporal range.

3.7 Feature Engineering and Classical Classifiers

Deep temporal features are complemented by explicit geometric and motion descriptors. These descriptors include joint positions, joint velocities, body proportions, head orientation, detection-box displacement, movement duration and statistics computed over a complete clip. Such features are useful in a small-data setting because they directly encode domain-relevant cues, such as prolonged head lowering during eating or sustained hand-to-dog proximity during petting.

The project uses Extremely Randomised Trees (ExtraTrees), linear Support Vector Machines (SVMs) and logistic regression for different feature branches. ExtraTrees averages a large collection of randomised decision trees and can represent non-linear thresholds in distance, duration and motion features. A linear SVM provides a complementary global decision boundary after feature standardisation. For high-dimensional visual features, Principal Component Analysis (PCA) is applied before logistic regression to reduce dimensionality and limit overfitting.

3.8 Ensemble Learning

Ensembling is used at three levels. First, classifier probabilities from complementary feature branches are combined by weighted soft voting or weighted log-probability fusion. Second, class-pair correction models refine decisions between selected confusable classes. Third, predictions from five cross-validation folds or five random-seed models are averaged at inference time. The saved fold weights therefore form one ensemble model rather than five separate model designs.

ExtraTrees is itself a **homogeneous ensemble**, because it combines many classifiers of the same type: randomised decision trees. Combining ExtraTrees with a linear SVM creates a **heterogeneous ensemble classifier**, since the two component models have different assumptions and decision functions. The complete interaction system further combines skeleton-relation and CLIP visual features; it can therefore be described more precisely as a heterogeneous ensemble with multi-branch feature fusion rather than simply as a hybrid machine-learning model.

The combination is also motivated by the bias–variance trade-off. Individual decision trees can learn non-linear thresholds and feature interactions but may have high variance on a small dataset. ExtraTrees reduces this variance through feature/threshold randomisation and probability averaging across 500 trees, potentially at the cost of a small increase in bias. In contrast, the regularised linear SVM has a simpler and more stable decision boundary but can have higher bias when the true class boundary is non-linear. Weighted soft voting is intended to retain the non-linear capacity of ExtraTrees while using the SVM as a stable complementary classifier. This is a design motivation rather than proof that variance has been reduced; the claim must be supported by controlled component comparisons.

3.9 Evaluation Methods

Model development uses five-fold grouped cross-validation. Four folds are used for training and the remaining fold for validation, so each development sample receives one prediction from a model that was not trained on that sample. Combining these predictions produces the out-of-fold (OOF) result. OOF Accuracy, Macro-F1 and the confusion matrix are the primary evidence used for model comparison.

A small held-out test set is also evaluated after model development. Because the number of test clips is limited, its results are treated as supporting evidence that the end-to-end approach is feasible rather than as the main estimate of generalisation performance.

4. System Design

4.1 Overall System Architecture

The system has three layers: a mobile web front end, a FastAPI back end and a Python inference pipeline. A clip passes through them as follows:

1. **Upload.** The user selects a video in the mobile web application, together with its recording time and camera name. The application posts it to POST /api/clips. The back end stores the file under its own clip identifier and creates an analysis job.
2. **Job queue.** A background worker analyses one clip at a time. While the job runs, the front end polls GET /api/jobs/{job_id}. The job reports four stages (detect, keypoints, behaviour and interaction) with counts of the work done in each.
3. **Windowing.** The inference entry point predict_video() (tools/predict_video.py) decides how to split the clip:
   - a clip of up to eight seconds is analysed as one window;
   - a longer recording is read lazily and divided into four-second windows, so that it never has to be held in memory as a whole.
4. **Skeleton extraction (Model One).** Each window passes through the dual-species pipeline of Section 4.2, which produces a two-stream skeleton array with detection boxes.
5. **Recognition (Models Two and Three).** Each window is routed by the fraction of frames in which the dog and the person were detected. The threshold is 50% of frames.

- **Dog only.** If the dog passes the threshold and the person does not, the dog stream is classified by the five-fold dog-behaviour ensemble (Section 5.2).
- **Dog and person.** If both pass, both streams and eight sampled RGB frames are classified by the interaction ensemble (Section 5.3). The behaviour label of the window is suppressed, because Model Two was trained only on clips without a person.
- **Neither case.** The window is marked as insufficient detection instead of receiving a label.

6. **Result.** The window probabilities are lightly smoothed over time, and neighbouring windows with the same label are merged into timeline segments. The output is written as result.json, with the window probabilities and merged timelines, and as timeline.csv, with one row per segment. Both files are stored next to the uploaded clip.
7. **Presentation.** The front end reads the results through GET /api/clips and shows them as a daily story, a per-clip result page and a weekly summary (Section 4.4).

The same predict_video() function is also available as a command-line tool, so the back end, the command line and the tests all exercise the same inference path.

4.2 Dual-Species Skeleton Extraction Pipeline

The skeleton extraction stage converts every video clip into the two-stream skeleton tensor used by both recognition models. It is shared by training and inference, so the training skeletons and the skeletons seen by the deployed system are produced in exactly the same way. The pipeline contains five steps:

1. **Frame sampling.** The clip is decoded and sampled at 15 fps.
2. **Whole-clip detection.** RTMDet-m is applied to every frame. Only person boxes and merged quadruped boxes are kept (Section 3.2).
3. **Per-species tracking.** One person track and one dog track are selected with the bidirectional rule of Section 3.4. Dog-behaviour clips track only the dog.
4. **Pose estimation.** The human RTMPose-m model (COCO-17) is applied in the person box and the dog RTMPose-m model (AP-10K-17) in the dog box; confidences are clipped to $[0.001,1]$.
5. **Output.** The two streams are stored together as one $2 \times T \times 17 \times 3$ array, with zeros for frames in which a target was not found.

The dog-behaviour model reads stream 1 only, while the interaction model reads both streams. Because the human branch uses the official model unchanged, all model-specific work in this stage concerns the dog detector post-processing, the tracking rule and the choice of dog pose weights (Section 5.1).

4.3 Repository and Module Structure

The repository separates reusable library code, command-line tools, the web service and the front end. Source videos, generated skeletons and the official pose weights are not stored in the repository. They are located through the PET_DATA_ROOT environment variable, and the clips and skeletons are published as a Hugging Face dataset (Appendix C).

Path	Contents
pet_recognition_system/pet_recognition/models/	rtmdet.py (detector wrapper, quadruped merging, box selection), rtmpose_onnx.py (ONNX pose inference), skeleton_tcn.py (motion-aware TCN)

pet_recognition_system/pet_recognition/inference/	skeleton.py (detection, tracking and two-stream skeleton extraction), video_pipeline.py (windowing, routing, smoothing and timeline merging), interaction_visual.py (frozen CLIP encoder)
pet_recognition_system/pet_recognition/preprocessing/	Interpolation, normalisation, global motion, temporal sampling, augmentation and relation features (Section 3.5)
pet_recognition_system/pet_recognition/datasets/	Behaviour and interaction manifest readers, grouped split assignment, LabelMe pose loader
pet_recognition_system/pet_recognition/evaluation/	Classification metrics and confusion matrices; OKS, AP and PCK for pose
pet_recognition_system/pet_recognition/visualization/	Skeleton overlay drawing for review videos
pet_recognition_system/tools/	Command-line entry points: pose evaluation and fine-tuning, skeleton extraction, cross-validation, final training, evaluation and predict_video.py
pet_recognition_system/app/server.py	FastAPI service with a background job worker
pet_recognition_system/configs/	MMPose configuration for P1 fine-tuning
pet_recognition_system/checkpoints/best_models/	Released dog-behaviour and interaction ensembles, model card and SHA-256 checksums
pet_recognition_system/outputs/best_models/	Frozen split manifests, OOF and test metrics, confusion matrices
pet_recognition_system/tests/	pytest tests for the API, the behaviour pipeline, skeleton extraction, keypoint metrics, box selection and the video pipeline
frontend/	React and Vite mobile web application, mock API and design prototype
diagrams/, presentation/, proposal/	Report figures, slides and the project proposal

The two recognition tasks are released as one ensemble each:

- **Dog behaviour.** Five TCN fold checkpoints plus the geometry-fusion and specialist joblib files.
- **Interaction.** Five relation-ensemble and five CLIP-branch joblib files.

The fold files are components of a single model, not alternative versions.

4.4 Frontend Prototype

The front end is a mobile web application built with React 18, React Router and Vite (Section 3.1). It has five pages, reached through a bottom navigation bar.

Page	Route	What it shows
Story	/	The activity review for one day. A swipeable deck of clip covers; a 06:00–22:00 day line with a mark for every clip; summary tiles for **Rest** (lying and sitting), **Active** (walking, standing and eating) and **With you** (feeding, petting and playing). Filters show all clips, clips with the dog alone, or clips with a person.

Upload	/upload	A file picker for MP4 or MOV files of up to 500 MB, with the recording date and time (pre-filled from the file) and a camera name (living room, kitchen, bedroom or garden). Input is validated before submission.
Analyze	/analyze/:jobId	Live progress of the job, polled every second. It lists the four stages (find dog and owner, trace 17 keypoints, recognise behaviour, read the interaction), with the state and progress count of each, and links to the result when the job finishes.
Result	/clips/:id	The clip cover and three cards. **The dog is** shows the top behaviour with probability bars averaged over windows, or "not scored" when a person was in view throughout. **With you** shows the interaction probabilities, or states that the dog was alone. **Timeline** lists the merged segments with clock times and confidence.
Week	/week	A stacked bar chart of daily minutes per behaviour from Monday to Sunday, the dominant behaviour of the week, and a "How sure are we?" card with each model's test accuracy and number of test clips, read from /api/models/metrics.

All data access goes through one module, src/api.js, which calls the REST endpoints and turns server errors into readable messages. Setting VITE_MOCK=1 replaces it with a mock API that serves sample clips with the same result format and simulates the four analysis stages. The interface can therefore be developed and demonstrated without the models. The logic that turns results into the numbers on screen (dominant label, day totals, weekly rows and averaged probabilities) lives in src/lib/summary.js and is covered by Vitest tests.

5. Models

5.1 Model One: Dog Pose Estimation

Model One produces the dog skeletons used by both recognition models. The human branch uses the official RTMPose-m COCO model without modification; only the dog branch was considered for adaptation, because public animal pose data contain few indoor home-camera scenes. Three variants were planned:

Variant	Dog pose weights	Status
P0	Official RTMPose-m AP-10K, no further training	Evaluated; **selected**
P1	P0 fine-tuned on the APT-36K dog subset	Trained and evaluated
P2	Fine-tuned on hand-labelled indoor frames	Not carried out

The variants are compared on the 36-frame indoor evaluation set using the real pipeline, in which the pose model receives RTMDet boxes rather than hand-drawn boxes (Section 6.3).

5.1.1 Baseline (P0: RTMPose-m AP-10K)

P0 is the official MMPose RTMPose-m checkpoint for AP-10K, pre-trained on AIC and COCO and then trained for 210 epochs on AP-10K, which covers 54 animal species. Its published results on the AP-10K test set are 0.722 AP, 0.939 AP50 and 0.788 AP75; for large animals the AP is 0.728. Before any fine-tuning, P0 reaches 0.775 AP (0.940 AP50, 0.882 AP75) on the APT-36K dog validation split. P0 is used without any training and serves both as the baseline and as the starting point for P1.

5.1.2 Fine-Tuning on APT-36K (P1)

P1 continues training P0 on the APT-36K dog training split. The model structure, the KLDiscretLoss of the official configuration and the data augmentation are unchanged, and no layer is frozen. Because training

starts from converged weights, the learning rate is one tenth of the official value and the schedule is much shorter.

Setting	P1	Official AP-10K configuration
Epochs	40 planned; **stopped manually after 30**	210
Learning rate	4×10^{-4}	4×10^{-3}
Schedule	Linear warm-up for 50 iterations; cosine decay from epoch 20	Warm-up for 1,000 iterations; cosine decay in the second half
Optimiser	AdamW, weight decay 0.05	Same
Batch size	64 (27 iterations per epoch)	64 × 8 GPUs
Augmentation	Flip, half-body, scale 0.6–1.4, rotation ±80°, HSV jitter, blur, random occlusion	Same
Weight averaging	EMA	Same
Checkpoint selection	Highest AP on the APT-36K validation split, evaluated every epoch	Evaluated every 10 epochs

The configuration planned a lighter augmentation stage for epochs 31–40; it was not reached because training was stopped after epoch 30. Training ran on a laptop CPU (Intel Core Ultra 7 255H) and took about 67 minutes for 30 epochs, because no Windows MMCV build supported the CUDA 12.8 runtime required by the available RTX 5060 GPU.

On the APT-36K validation split, AP fell to 0.753 after the first epoch, exceeded P0 from epoch 3 and fluctuated between 0.80 and 0.82 after epoch 10 (Figure 6-1). The best value, 0.817, was reached at epoch 19 and this checkpoint is used as P1. The training loss kept decreasing from 0.458 to 0.277 until epoch 30, while validation AP no longer improved after about epoch 10; the later epochs therefore mainly fitted the APT-36K training videos. Since the validation split contains only three videos, the choice of epoch 19 over neighbouring epochs is partly due to chance.

5.1.3 Further Adaptation (P2)

P2 was intended to fine-tune the dog model on hand-labelled indoor frames from the project's own clips. It was not carried out, for three reasons. First, P1 did not improve indoor accuracy (Section 6.3), so additional data from a different public source was not the limiting factor. Second, P0 already performs close to its own benchmark on indoor dogs: its AP with hand-drawn boxes, 0.700, is within the confidence interval of the official large-animal result. Third, the evaluation frames now come from 28 source videos; a fair P2 would require a new video-level split in which none of these videos is used for training, leaving few videos for training.

P0 skeletons were therefore exported for the downstream models. P2 would become worthwhile if the behaviour or interaction models were shown to be limited by skeleton quality, particularly for lying dogs, which are the weakest posture for both detection and pose estimation.

5.2 Model Two: Dog Behaviour Recognition

The dog-behaviour model receives a temporal sequence of AP-10K 17-joint dog skeletons and predicts one of five classes: walking/running, standing, sitting, lying/sleeping, and eating/drinking. Every clip is converted to $T = 32$ frames. Each joint contains an x-coordinate, a y-coordinate and a confidence score, giving a basic input tensor

$$\mathbf{X} \in \mathbb{R}^{32\times17\times3}.$$

Missing keypoints are temporally interpolated where possible, coordinates are normalised using image and body scale, and a frame mask distinguishes observed frames from padded positions. Unlike a representation based only on frame-wise centred skeletons, the proposed input retains detection-box motion so that movement across the image is not discarded.

5.2.1 Motion-Aware TCN Backbone

At each time position, the network receives three groups of features:

1. 51 pose values from the 17 joints and their confidence scores;
2. 51 first-order temporal differences describing joint motion; and
3. six global motion values derived from the dog detection box, including centre displacement and scale change.

The resulting 108-dimensional vector is projected to 64 channels by a 1×1 convolution. It then passes through three residual TCN blocks with kernel size 3 and dilation rates 1, 2 and 4. All temporal convolutions use stride 1 and suitable padding; therefore, the sequence remains 32 frames long throughout the backbone rather than being temporally downsampled.

The frame-level features are aggregated by confidence-weighted temporal pooling:

$$\mathbf{h} = \frac{\sum_{t=1}^{T} w_t \mathbf{H}_t}{\sum_{t=1}^{T} w_t + \epsilon},$$

where w_t is the mean confidence of the 17 joints in frame t. During training, an auxiliary moving-versus-static head encourages the backbone to distinguish high-motion behaviours from stable postures. Only the five-class behaviour head is used during inference.

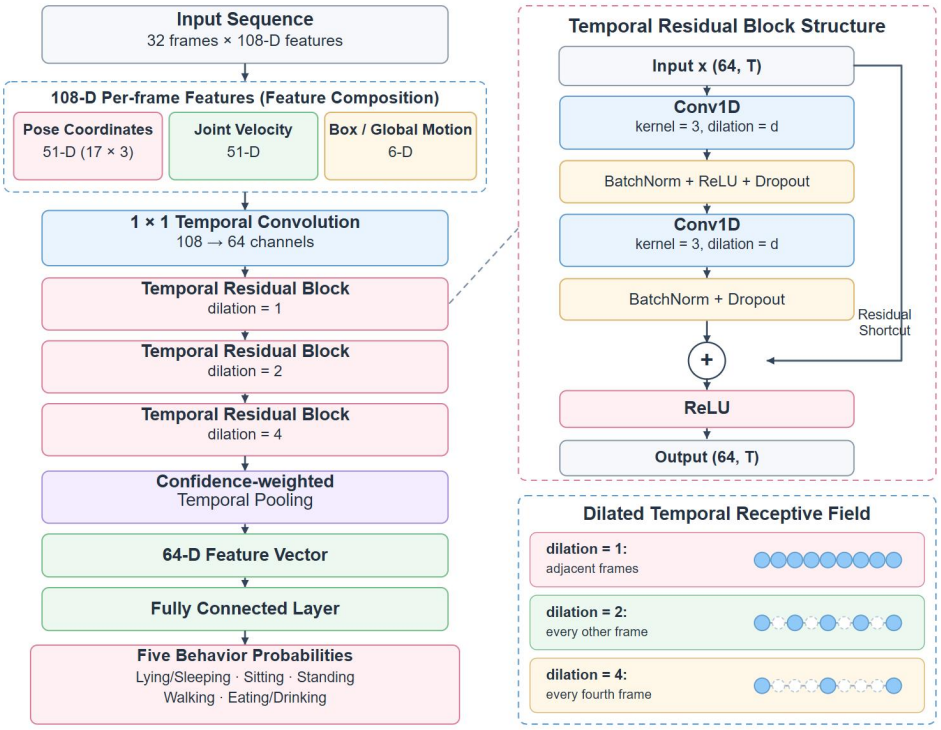


Figure 5-1. Architecture of the motion-aware TCN. Each 32-frame sequence contains 108 features per frame: 51 pose-coordinate channels, 51 joint-velocity channels and six detection-box/global-motion channels. A 1×1 temporal projection maps the input to 64 channels, followed by three residual temporal blocks with dilation rates 1, 2 and 4. Confidence-weighted temporal pooling produces a 64-dimensional clip representation for

five-class prediction. The right-hand panels detail the residual block and the increasing temporal receptive field.

5.2.2 Engineered Geometry Features

The dataset is relatively small, and several classes differ through cues that are difficult to learn reliably from the TCN alone. The system therefore constructs an additional 834-dimensional clip-level vector from the normalised AP-10K skeleton and the six global-motion channels. The vector contains four concatenated feature blocks:

Feature block	Dimension	Main information
General motion summary	160	Detection-box translation and scale change; whole-body and per-joint motion
Standing-oriented block	230	General motion plus paw support, limb extension, trunk orientation and stillness
Eating-oriented block	262	General motion plus head-lowering geometry, head motion and low-head duration
Walking-oriented block	182	General motion plus path persistence, speed, acceleration and paw dynamics
Concatenated vector	**834**	Input to the engineered-feature classifier

The feature blocks were originally designed for class-specific specialists and were later concatenated without changing their interfaces. Consequently, the 160-dimensional general-motion summary is present in the general block and is repeated inside the three class-oriented blocks. The 834 dimensions therefore represent the implemented concatenated vector rather than 834 unique physical quantities.

For any frame-level signal q_t, the main temporal summary is

$$S(q) = [\text{mean}(q), \text{std}(q), \min(q), \max(q), Q_{25}(q), Q_{50}(q), Q_{75}(q)].$$

The **160-dimensional general-motion summary** contains ten frame-level signals: the two-dimensional displacement of the detection-box centre from the first frame, its two-dimensional velocity, relative log-scale, scale velocity, displacement magnitude, centre speed, mean joint speed and maximum joint speed. Applying the seven statistics above gives 70 values. Five additional clip-level quantities describe accumulated centre and joint motion and the horizontal and vertical movement ranges. Finally, each of the 17 joints contributes its mean speed, maximum speed, 90th-percentile speed, spatial range and mean confidence, giving another $17 \times 5 = 85$ values. The total is therefore $70 + 5 + 85 = 160$.

(a) Standing-oriented geometry. The system uses the four AP-10K limb chains: shoulder–elbow–front paw and hip–knee–rear paw on the left and right sides. For each leg, an extension ratio is calculated as

$$r_{\text{leg}} = \frac{\| \mathbf{p}_{\text{proximal}} - \mathbf{p}_{\text{paw}} \|}{\| \mathbf{p}_{\text{proximal}} - \mathbf{p}_{\text{middle}} \| + \| \mathbf{p}_{\text{middle}} - \mathbf{p}_{\text{paw}} \|}.$$

The model also measures the vertical spread of the four paws, their position below the neck/tail-root body axis, the horizontal orientation of the neck-to-tail vector, visible-paw confidence, detection-box centre speed, and mean and maximum joint speed. These ten frame-level signals are each summarised by $S(\cdot)$, adding 70 values to the 160-dimensional general summary and producing the 230-dimensional standing block. A standing sequence is therefore represented by extended supporting limbs, relatively aligned paws, a near-horizontal trunk and low motion, rather than by a single static pose.

(b) Eating-oriented geometry. This block focuses on the nose (joint 2), neck (3), tail root (4), shoulders (5 and 8), front paws (7 and 10) and rear paws (13 and 16). Its main distances are the nose-to-nearest-front-paw distance, nose height relative to the four-paw ground estimate, nose drop relative to the neck and

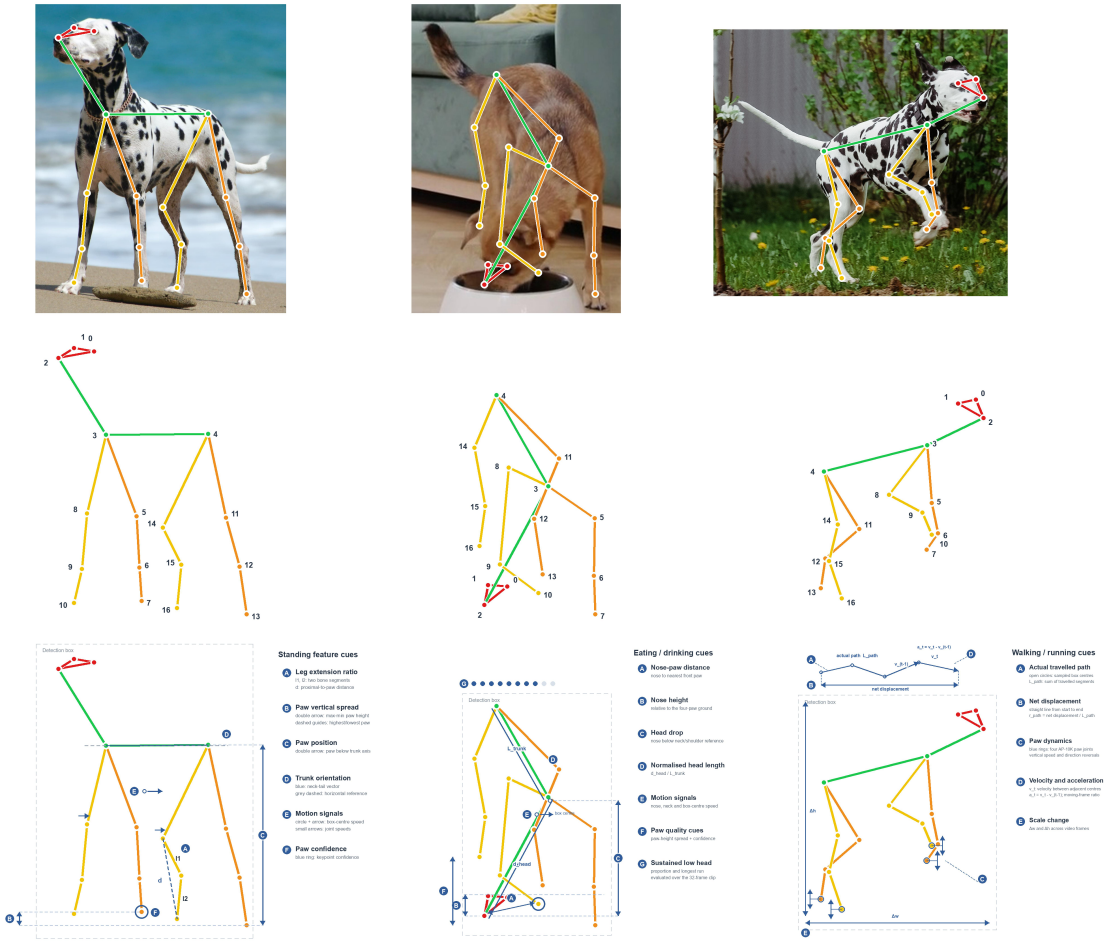
shoulder midpoint, and the nose-to-neck distance normalised by neck-to-tail trunk length. Nose speed, neck speed, centre speed, paw-height spread and relevant keypoint confidences complete 12 frame-level signals, which contribute $12 \times 7 = 84$ statistics. Twelve duration features measure the proportion and longest continuous run of low-head, strongly-low-head and low-head-with-low-body-motion frames, as well as head-drop percentiles. Six further values describe vertical head-direction changes, nose movement range, accumulated nose/centre motion and valid sequence length. Together with the general summary, this gives $160 + 84 + 12 + 6 = 262$ dimensions. These duration features distinguish sustained eating/drinking patterns from a brief downward glance.

(c) Walking-oriented geometry. This block adds 22 descriptors to the general summary. They include total centre-path length, net displacement and trajectory straightness,

$$r_{\text{path}} = \frac{\| \mathbf{c}_T - \mathbf{c}_1 \|}{\sum_{t=2}^{T} \| \mathbf{c}_t - \mathbf{c}_{t-1} \| + \epsilon},$$

together with speed and acceleration statistics, the proportion of moving frames, direction changes, four-paw speed and vertical-direction reversals, and detection-box scale variation. Walking/running is consequently supported by persistent global translation and coordinated paw dynamics rather than by large joint motion alone.

Thus, the engineered branch combines single-joint trajectories, distances and ratios between selected joints, whole-body detection-box motion, and temporal duration/statistical descriptors. The implementation does not calculate a comprehensive set of anatomical joint angles; “limb-extension ratios and trunk orientation” is a more precise description of the geometry used.



(a) Standing (b) Eating / drinking (c) Walking / running

Figure 5-2. Engineered geometry and motion cues used by the dog-behaviour classifier. The columns show (a) standing, (b) eating/drinking, and (c) walking/running. From top to bottom, each column presents the input image with the estimated AP-10K skeleton, the isolated skeleton with 0-based joint indices, and the corresponding engineered-feature illustration. The blue annotations identify the class-relevant geometry, motion, confidence, duration, trajectory and detection-box signals described in this section.

5.2.3 Geometry and Pairwise Fusion

For each fold, the TCN probability and the engineered-feature classifier probability are fused in log-probability space:

$$\mathbf{z} = \log(\mathbf{p}_{\mathrm{TCN}} + \epsilon) + \alpha \log(\mathbf{p}_{\mathrm{geom}} + \epsilon).$$

Log-probability fusion allows the geometry branch to adjust the temporal model while retaining the TCN as the principal sequence encoder. The fused logits are converted back to class probabilities before class-pair refinement.

5.2.4 Class-Specialist Models

Two ExtraTrees pairwise specialists refine class pairs that remained difficult in the cross-validation confusion matrix. The first specialist distinguishes standing from eating/drinking. It applies a low-weight correction ($\alpha = 0.1$) to this pair for every sample, while leaving the other three class logits unchanged. The second

specialist distinguishes eating/drinking from sitting and applies a stronger correction ($\alpha = 0.5$) only when these two classes occupy the top two candidate positions. Each specialist converts its binary probability into a smoothed log-odds value, adds the weighted value to the positive-class logit, and subtracts it from the negative-class logit. This design focuses additional capacity on the two difficult decision boundaries without replacing the original five-class prediction.

The complete dog-behaviour system contains five models produced by five-fold cross-validation. Their output probabilities are averaged during inference. The five stored weights are components of one final ensemble, not five competing behaviour models.

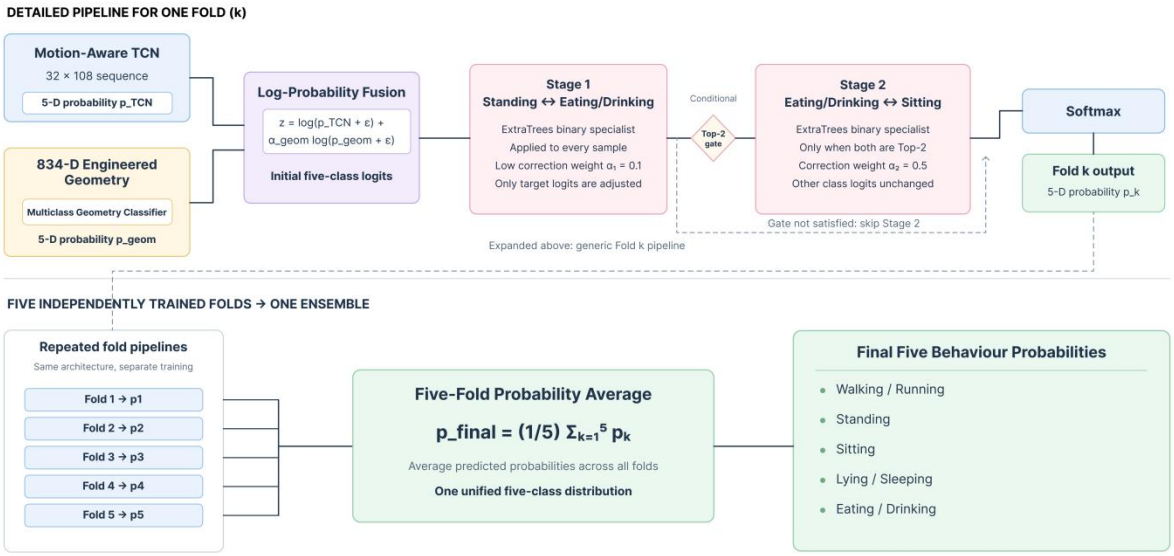


Figure 5-3. Final dog-behaviour ensemble. Within each fold, the Motion-Aware TCN probabilities are fused with the 834-dimensional engineered-geometry classifier in log-probability space. Two ExtraTrees specialists then refine the standing–eating/drinking and eating/drinking–sitting boundaries, with the second correction enabled only when the relevant classes occupy the top two positions. Softmax probabilities from the five independently trained folds are averaged to produce the final five-class prediction.

5.3 Model Three: Human-Dog Interaction Recognition

The interaction model predicts no interaction, hand feeding, petting, or playing together from synchronised COCO-17 human skeletons, AP-10K-17 dog skeletons, detection boxes and selected RGB frames. The design separates interpretable skeleton-relation evidence from complementary visual context.

5.3.1 Human-Dog Relation Features

For each frame, the system extracts a 48-dimensional relation vector from the synchronised human skeleton, dog skeleton and two detection boxes. These features preserve cross-subject geometry that would be lost if the two skeletons were normalised independently. They are organised into five groups:

- **Global person-dog geometry:** relative box-centre displacement, centre distance, relative velocity and closing speed.
- **Hand-to-mouth geometry:** vectors and distances from both COCO wrists to AP-10K joint 2. Because AP-10K has no mouth landmark, the dog's nose is used as a stable mouth proxy. Contact distances are divided by the dog-box diagonal so that their scale remains comparable across near and distant views.

- **Arm-to-trunk contact:** minimum distances from the wrists, elbows, sampled arm segments and human torso to the dog's trunk graph, together with continuous contact scores and binary contact indicators.
- **Posture and contact context:** human and dog orientation, keypoint confidence, detection-box overlap, relative stillness, arm-crossing evidence and an approximate embrace score.
- **Interaction motion:** human and dog box-centre speed, mean and maximum skeleton speed, wrist and elbow speed, acceleration, single-subject activity, joint activity and a playing-motion score.

For a 32-frame clip, every relation channel is summarised by its mean, standard deviation, minimum, maximum, and 25th, 50th and 75th percentiles, producing $48 \times 7 = 336$ values. These order-independent statistics describe the distribution of each relation but cannot represent whether a hand first approaches, remains near and then withdraws from the dog's mouth. The system therefore adds 16 ordered descriptors for hand feeding. They encode the longest near/contact run, stable-contact proportion, start/minimum/end hand-nose distance, approach and withdrawal magnitude, whether the active hand is closer to the nose than to the trunk, dominant-hand consistency, hand-switch frequency, extreme closing and withdrawal speed, and an approach-duration interaction term. The final relation vector has

$$48 \times 7 + 16 = 352$$

dimensions.

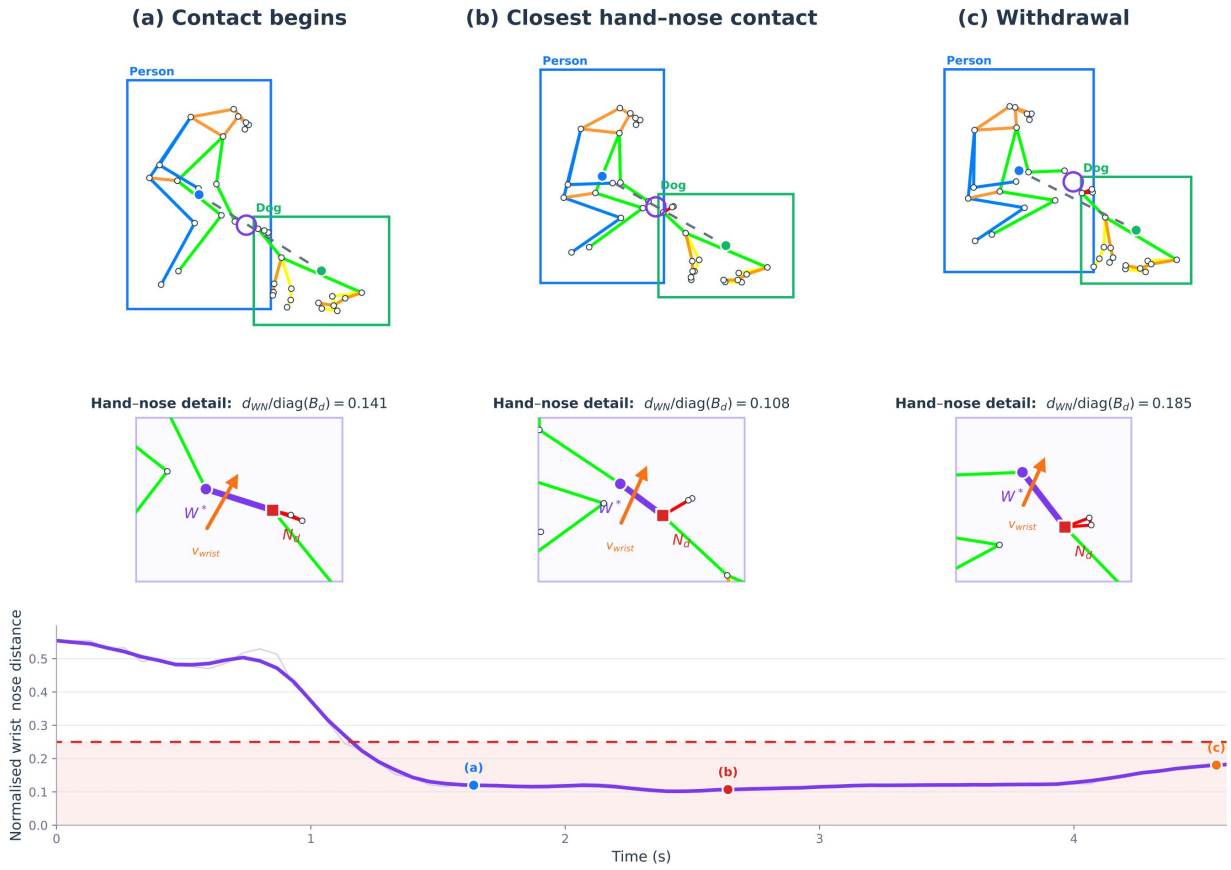


Figure 5-4. Hand-feeding example generated from the actual RTMDet and RTMPose outputs for clip d50_p52_v01_c001. The upper row shows the human COCO-17 skeleton, dog AP-10K skeleton and their detection boxes at (a) contact onset, (b) the closest hand–nose contact and (c) withdrawal onset. The middle row enlarges the nearest human wrist W^* and dog nose N_d. Their scale-normalised distance is $d_{WN}/\mathrm{diag}(B_d)$, where B_d is the dog detection box; the three selected frames yield 0.141, 0.108 and 0.185, respectively. The

orange arrow denotes wrist displacement direction between the selected frames. The lower plot shows the frame-level distance over the complete clip together with a five-frame smoothed curve. The red dashed line is the implemented contact threshold of 0.25, and the shaded area denotes the contact region. Skeleton links use the official MMPose COCO-17 and AP-10K colour conventions.

5.3.2 ExtraTrees / Linear-SVM Soft-Vote Ensemble

The relation branch combines an ExtraTrees classifier containing 500 decision trees with a linear SVM. Each tree learns non-linear decisions using relation features such as contact distance, duration and activity, while the linear SVM learns a complementary global boundary on the standardised 352-dimensional vector. Their probabilities are combined by soft voting:

$$\mathbf{p}_r = 0.8\mathbf{p}_{\mathrm{ET}} + 0.2\mathbf{p}_{\mathrm{SVM}}.$$

Five classifiers are trained with random seeds 29–33, and their probabilities are averaged to obtain the final relation-branch probability $\mathbf{p}_r$.

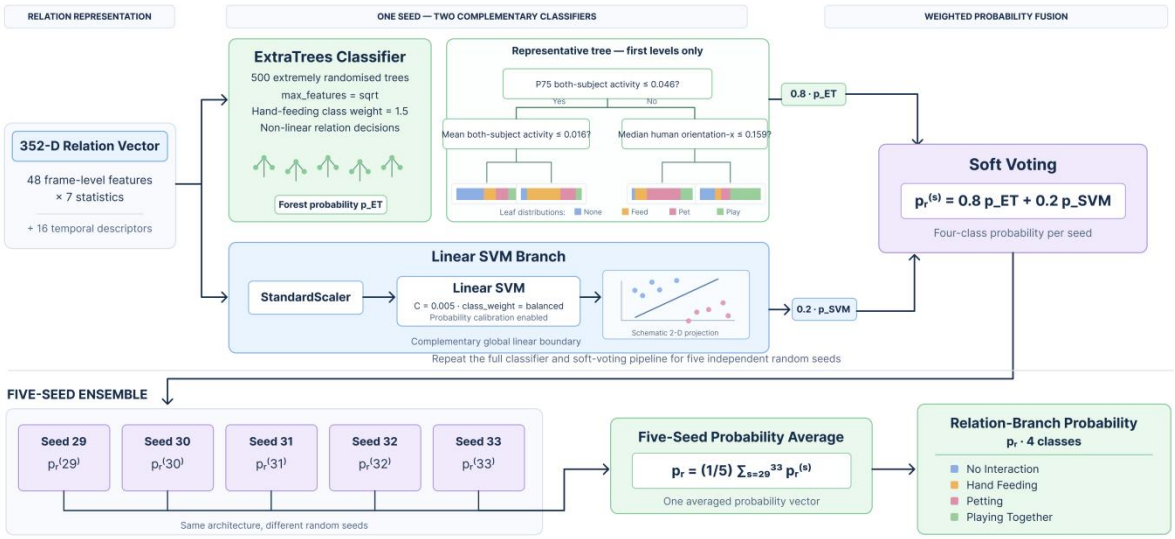


Figure 5-5. Relation-branch ensemble for human–dog interaction recognition. The 352-dimensional relation vector is processed by a 500-tree ExtraTrees classifier and a standardised linear SVM. Their class probabilities are combined using weights of 0.8 and 0.2, respectively. The complete classifier is trained with five random seeds, and the resulting probability vectors are averaged to obtain the final four-class relation prediction. The representative tree shows only its first decision levels.

5.3.3 Frozen CLIP Visual Branch

Skeletons do not explicitly show food, toys or whether physical contact has actually occurred. A frozen CLIP ViT-B/32 encoder is therefore used as an auxiliary visual branch. Its parameters are not fine-tuned, reducing the risk of overfitting on the small interaction dataset.

Eight frames are sampled uniformly from each clip. For every sampled frame, the system generates both a union crop covering the person and dog and a full-scene image, producing 16 images in total. CLIP encodes each image as a 768-dimensional vector. For the crop sequence and full-scene sequence separately, the mean, standard deviation and absolute first-to-last difference are calculated, giving

$$2 \times 3 \times 768 = 4608$$

visual features. These features are standardised, reduced to 32 dimensions by PCA and classified by logistic regression. The probabilities of five visual models are averaged to obtain $\mathbf{p}_v$.

5.3.4 Probability-Level Fusion

The relation branch remains the main source of evidence, while visual information is assigned a smaller weight. The final prediction is

$$\mathbf{p} = \text{softmax}[\log(\mathbf{p}_r + \epsilon) + 0.1\log(\mathbf{p}_v + \epsilon)].$$

This fusion allows food, toy and contact appearance to refine the relation prediction without allowing the small visual dataset to dominate the decision.

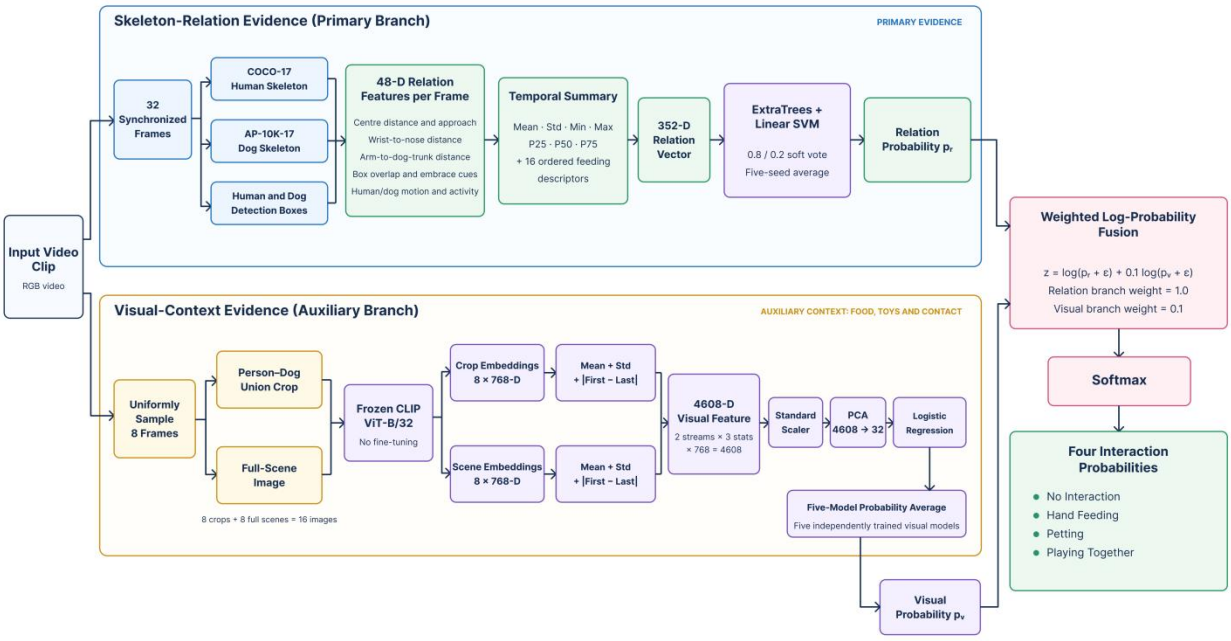


Figure 5-6. Final dual-branch human–dog interaction model. The primary branch summarises synchronised human and dog skeletons and detection boxes into a 352-dimensional relation vector, which is classified by the ExtraTrees/linear-SVM ensemble. The auxiliary branch extracts frozen CLIP ViT-B/32 features from eight person–dog union crops and eight full-scene images, aggregates them into a 4,608-dimensional representation, and applies standardisation, 32-dimensional PCA and logistic regression. The five-model visual probability is combined with the relation probability through weighted log-probability fusion, using weights of 1.0 and 0.1 respectively, before the final four-class Softmax output.

5.4 Alternative Models Explored

LSTM, ST-GCN and Transformer-based skeleton classifiers were explored during development. The LSTM served as a recurrent sequence baseline. ST-GCN explicitly represented joints as graph nodes and physical connections as edges, while the Transformer used attention to model long-range temporal dependencies. These alternatives were valuable for identifying modelling requirements, but the final dog-behaviour system uses the motion-aware TCN and engineered-feature ensemble because it provided the strongest reproducible five-fold result under the current data constraints.

For interaction recognition, skeleton-only temporal and graph models were also considered. However, the clearest interaction cues were explicit cross-subject relations such as hand-to-mouth distance, contact duration and relative activity. The final design therefore uses a relation-feature ensemble, supplemented by frozen visual features. Comparable numerical results for alternative models should only be reported after all methods have been rerun on the same grouped five-fold split.

6. System Performance

6.1 Experimental Setup

All splits were created before augmentation and feature generation. Clips from the same source video were kept in one partition, and dog and person identities were separated between the development data and held-out test data where possible.

The dog-behaviour dataset contains 170 valid project clips. Of these, 33 clips form a small held-out test set and the remaining 137 clips are used for five-fold grouped cross-validation. A further 31 curated external APT skeleton sequences are used only in the training portion of each fold and never in validation or testing. The 33 test clips contain 6 walking/running, 8 standing, 6 sitting, 7 lying/sleeping, and 6 eating/drinking examples.

For interaction recognition, 75 training clips and 22 validation clips are combined into a 97-clip development set for five-fold grouped OOF evaluation. A separate 24-clip test set contains six examples of each interaction class and has no recorded identity overlap with the development set.

In each cross-validation round, four folds are used for training and one fold is used for validation. After all five rounds, every development clip has exactly one prediction from a model that did not train on that clip. These predictions are concatenated to compute the OOF metrics. Because the held-out test sets are small, OOF Accuracy and OOF Macro-F1 are treated as the primary results; test-set results are reported only as supporting evidence of end-to-end feasibility.

6.2 Evaluation Metrics

Accuracy is the proportion of correctly classified clips:

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N_{\text{all}}}.$$

Because the class counts are not perfectly balanced, Macro-F1 is also reported. It first calculates the F1 score independently for every class and then gives all classes equal weight:

$$\text{Macro-F1} = \frac{1}{C}\sum_{c=1}^{C} \text{F1}_c\,.$$

Per-class precision, recall and F1, together with confusion matrices, are used for error analysis. Recall is particularly useful for showing which behaviours are missed, but the small number of clips per test class means that each test error changes the percentage substantially.

6.3 Dog Pose Estimation Results

The dog pose variants are evaluated on the 36-frame indoor evaluation set (Section 2.4). The primary setting follows the real pipeline: RTMDet detects the dog and the pose model predicts keypoints inside the detected box. Running the pose models inside the hand-drawn boxes is reported as a reference. P1 uses its epoch-19 checkpoint. Both models are exported to ONNX by the same code; re-exporting P0 this way reproduces the official ONNX output exactly, so the two models share identical pre- and post-processing. No flip test is used.

APT-36K validation. Fine-tuning improves AP on the APT-36K validation split from 0.775 to 0.817 (Figure 6-1).

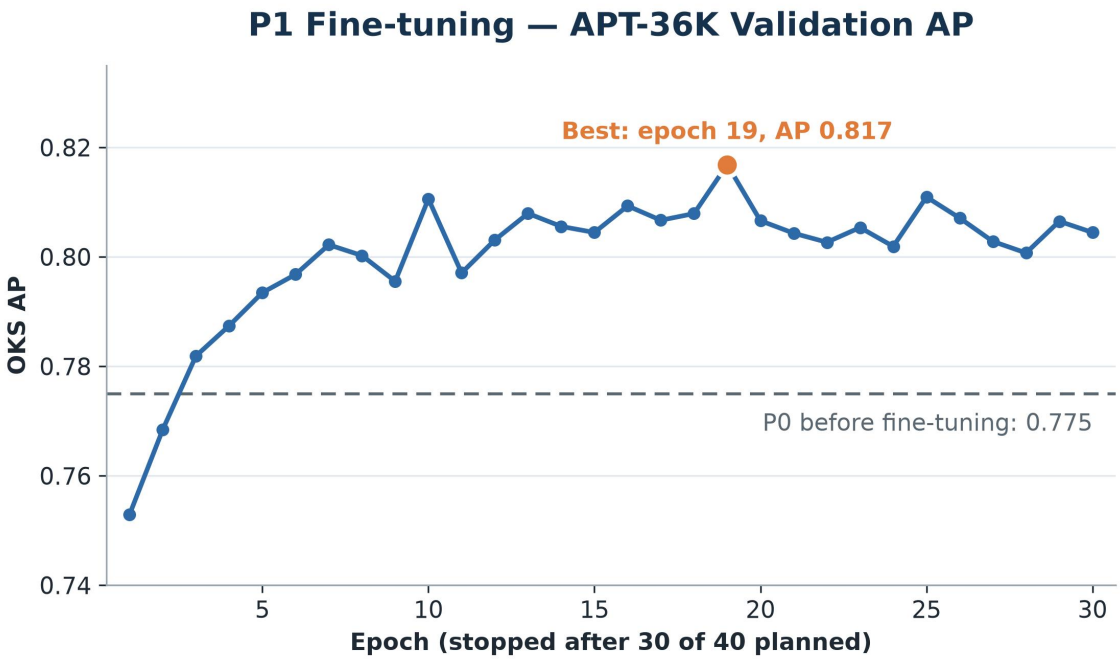


Figure 6-1. OKS AP of P1 on the APT-36K dog validation split after each epoch. The dashed line marks P0 before fine-tuning (0.775). AP exceeds P0 from epoch 3, levels off between 0.80 and 0.82 after epoch 10 and peaks at 0.817 at epoch 19, the checkpoint used as P1. Training was stopped after epoch 30 of the 40 planned.

Indoor evaluation. The gain on APT-36K does not carry over to the indoor frames.

Pose-model input box	Model	OKS AP	AP50	AP75	Mean OKS	PCK@0.05	PCK@0.1
RTMDet box (real pipeline)	P0	0.678	0.970	0.762	0.827	0.682	0.857
RTMDet box (real pipeline)	P1	0.693	1.000	0.754	0.831	0.680	0.861
Hand-drawn box (reference)	P0	0.700	0.970	0.825	0.839	0.695	0.868
Hand-drawn box (reference)	P1	0.698	1.000	0.758	0.832	0.654	0.857

Because the same 36 frames are scored by both models, the difference is tested with a paired bootstrap: the per-frame change in OKS (P1 minus P0) is resampled over frames 10,000 times.

Pose-model input box	Mean OKS change	95% confidence interval	Improved	Within ±0.01	Worse
RTMDet box	+0.004	−0.024 to +0.032	17	7	12
Hand-drawn box	−0.007	−0.028 to +0.013	11	11	14

Both intervals include zero, and the sign of the mean change differs between the two box sources. P1 and P0 are therefore tied on indoor frames. Individual frames move in both directions by large amounts, from −0.29 to +0.24 (Figure 6-4), which indicates that fine-tuning changed the model's behaviour on individual dogs without making it better on average. The detection step costs little accuracy: with P0, mean OKS falls from 0.839 with hand-drawn boxes to 0.827 with RTMDet boxes.

By behaviour. Lying/sleeping is the weakest posture for both models, with a mean OKS about 0.1 lower than the other labels (Figure 6-2). The remaining differences between P0 and P1 go in both directions and, with six frames per label, are within noise.

Behaviour label	Frames	P0 mean OKS	P1 mean OKS	Change	P0 PCK@0.1	P1 PCK@0.1
All	36	0.827	0.831	+0.004	0.857	0.861
Walking / Running	6	0.799	0.853	+0.054	0.750	0.837
Standing	6	0.915	0.858	−0.056	0.950	0.920
Sitting	6	0.866	0.844	−0.021	0.868	0.831
Lying / Sleeping	6	0.705	0.734	+0.029	0.802	0.779
Eating / Drinking	6	0.847	0.866	+0.019	0.927	0.948
Playing alone	6	0.831	0.831	0.000	0.833	0.833

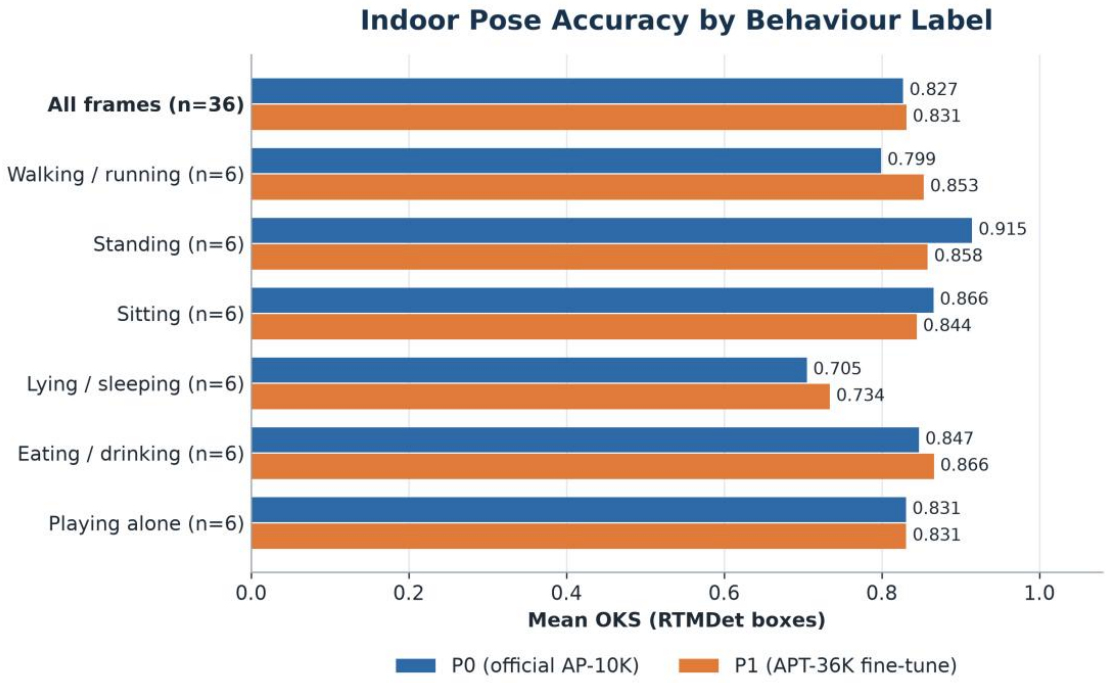


Figure 6-2. Mean OKS of P0 and P1 for each behaviour label, using RTMDet boxes. Lying/sleeping is the lowest for both models; differences between the models in the other labels are small relative to the six frames per label.

By keypoint. Neither model dominates across keypoints (Figure 6-3). P1 is slightly better on the head, neck and front legs and slightly worse on the tail root, right hip and back legs. Both models are weakest on joints that are often partly hidden by the body: the right shoulder (PCK@0.1 of 0.686 for P0 and 0.771 for P1), the left hip (0.714 for both) and the right hip (0.788 and 0.758). Each keypoint has only 26–36 labelled instances, so a difference of about 0.03 corresponds to a single point.

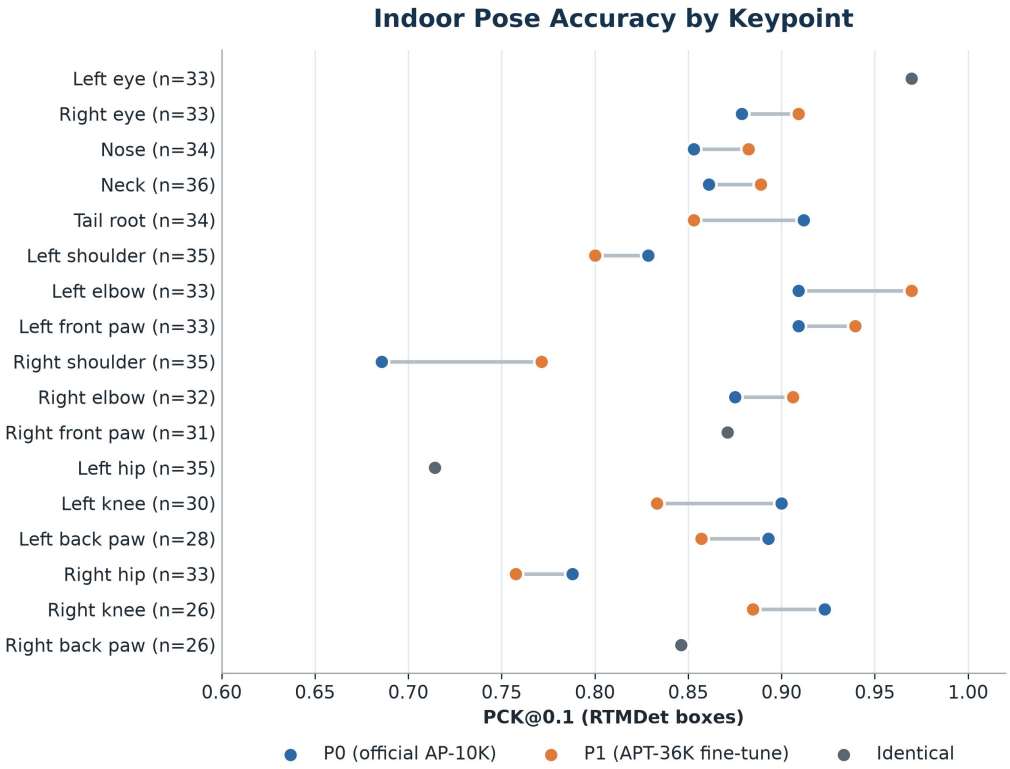


Figure 6-3. PCK@0.1 of P0 and P1 for each of the 17 AP-10K keypoints, using RTMDet boxes. Grey markers indicate keypoints on which the two models are identical. The shoulders and hips are the weakest joints for both models.

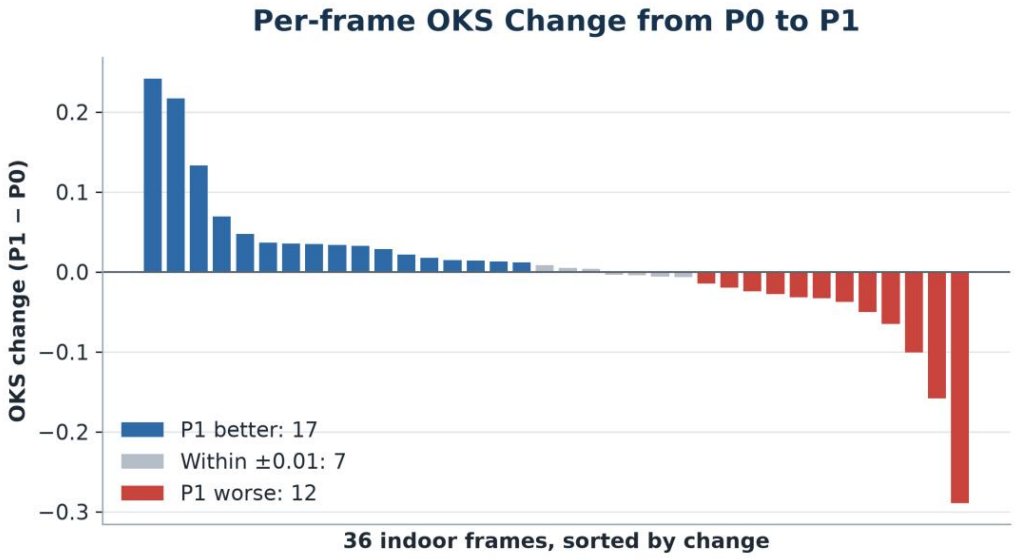


Figure 6-4. Per-frame OKS change from P0 to P1 on the 36 indoor frames, using RTMDet boxes and sorted by change. Seventeen frames improve, twelve become worse and seven change by less than 0.01; the largest changes are −0.29 for a standing dog and +0.24 for a lying dog.

Comparison with the official benchmark. The most comparable official figure is the AP-10K large-animal AP, because the official evaluation also uses ground-truth boxes and the indoor dogs occupy a large part of the frame.

Evaluation data	AP	AP50	AP75
Official: AP-10K test set (54 species)	0.722	0.939	0.788
Official: AP-10K test set, large animals	0.728	–	–
Indoor, hand-drawn boxes	0.700	0.970	0.825
Indoor, RTMDet boxes	0.678	0.970	0.762

The 95% bootstrap interval of the indoor AP (2,000 resamples over frames) is 0.623–0.775 with hand-drawn boxes and 0.590–0.760 with RTMDet boxes. The official large-animal AP of 0.728 lies inside the first interval, so there is no clear gap between P0's indoor performance and its own benchmark. This comparison is approximate in both directions: the indoor set scores only visible joints, excludes severely blurred frames and contains one large dog per frame, which favours the indoor result, while the official evaluation uses flip testing, which typically adds 0.5–1 AP point and was not used indoors. The official results are also reported for all species together rather than for dogs alone.

Decision. P0 is retained as the dog pose model. The APT-36K improvement of about four AP points did not transfer to indoor home-camera scenes, and P0 is already close to its benchmark level. These results have several limitations: the evaluation set has 36 frames and one annotator; occluded joints are not evaluated, although RTMPose still outputs them to the downstream models (with a median confidence of 0.46, compared with 0.79 for labelled joints); frames without a dog, scenes with a real cat and the person boxes were not evaluated; and only one fine-tuning configuration was tried.

6.4 Dog Behaviour Recognition Results

The primary five-fold OOF result of the final dog-behaviour ensemble is **77.37% Accuracy** and **76.47% Macro-F1**. This is the main quantitative estimate reported for model comparison. On the additional 33-clip test set, the model obtains 78.79% Accuracy and 78.83% Macro-F1. The test result supports the feasibility of the approach but is not used as the principal performance claim because of the limited sample count.

Evaluation	Clips	Accuracy	Macro-F1	Role in the report
Five-fold OOF	137 project clips	**77.37%**	**76.47%**	Primary result
Small held-out test	33	78.79%	78.83%	Feasibility evidence

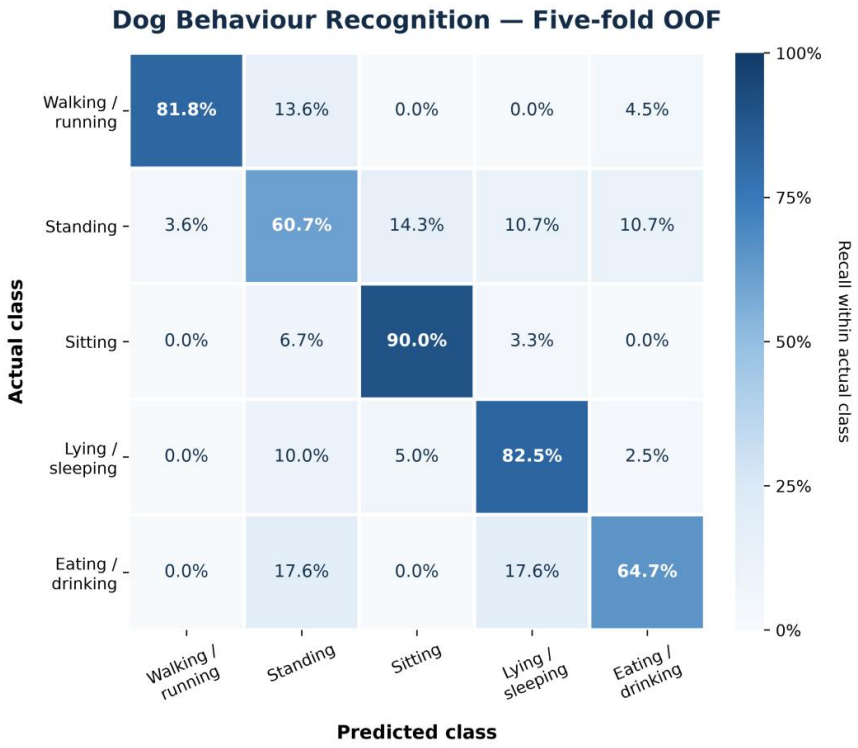


Figure 6-5. Five-fold out-of-fold confusion matrix for the final dog-behaviour model. Each cell reports the percentage within its actual class. The model is strongest on sitting, lying/sleeping and walking/running, while standing and eating/drinking remain the principal sources of confusion.

The test-set recalls are shown below for supplementary class-level analysis.

Dog behaviour class	Test clips	Correct	Recall
Walking / Running	6	5	83.33%
Standing	8	7	87.50%
Sitting	6	5	83.33%
Lying / Sleeping	7	6	85.71%
Eating / Drinking	6	3	50.00%

6.5 Human-Dog Interaction Recognition Results

The interaction experiments compare the skeleton-relation branch, the frozen CLIP visual branch and their final fusion.

Method	OOF Accuracy (primary)	OOF Macro-F1 (primary)	Small-test Accuracy (supporting)	Small-test Macro-F1 (supporting)
Relation branch	77.32%	75.89%	70.83%	69.36%
CLIP visual branch	60.82%	60.78%	75.00%	73.40%
Relation + CLIP ($\alpha = 0.1$)	**77.32%**	**76.13%**	**83.33%**	**83.30%**

The relation branch is substantially stronger than the visual branch in five-fold OOF evaluation and is therefore the principal component of the interaction model. Adding CLIP leaves OOF Accuracy unchanged and increases OOF Macro-F1 slightly from 75.89% to 76.13%. The higher result on the 24-clip test set indicates that appearance may help with food, toys and physical-contact cues; because the test set is small, this result is interpreted as evidence that the fusion scheme is feasible rather than as the main generalisation estimate.

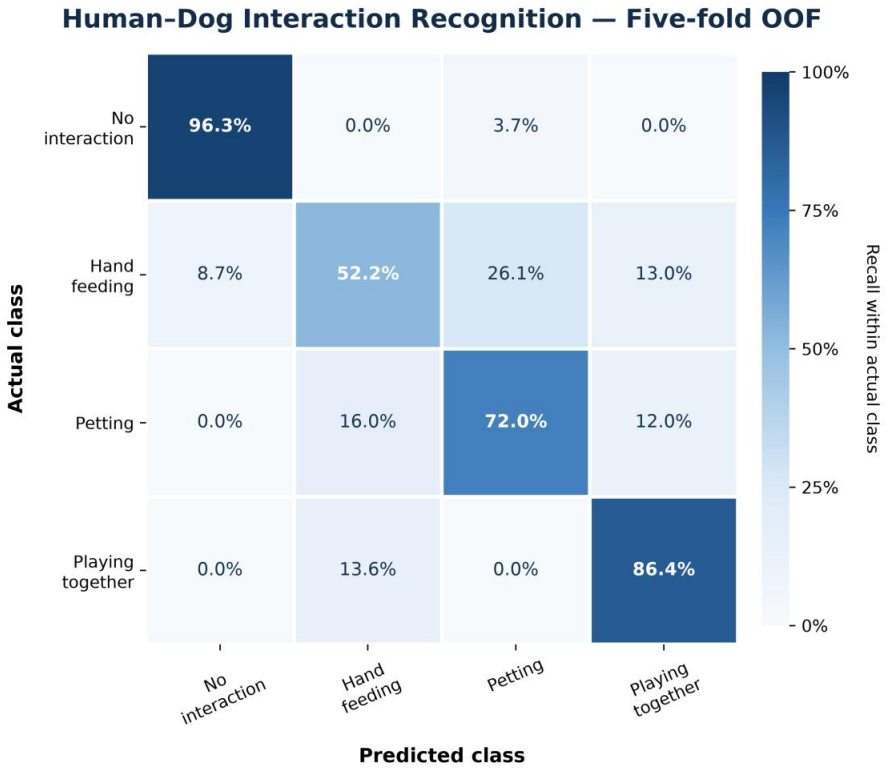


Figure 6-6. Five-fold out-of-fold confusion matrix for the final relation-plus-CLIP interaction model. Each cell reports the row-normalised percentage. No interaction and playing together have the highest recall; hand feeding is most often confused with petting, reflecting their shared close hand-to-dog geometry.

The following test recalls are included only as supplementary observations.

Interaction class	Test clips	Correct	Recall
No Interaction	6	5	83.33%
Hand Feeding	6	5	83.33%
Petting	6	4	66.67%
Playing Together	6	6	100.00%

6.6 Ablation Studies

The dog-behaviour ablation follows the construction of the final system incrementally. All three variants use the same 137 OOF clips and the same grouped five-fold split; the 31 curated external APT sequences are available only to the training portion of each fold. The base Motion-Aware TCN already contains joint velocity, global detection-box displacement and confidence-weighted temporal pooling. The next stage adds the 834-dimensional clip-level geometry classifier, and the final stage adds the two conditional pairwise corrections.

Dog-behaviour variant	Component added at this stage	OOF Accuracy	OOF Macro-F1	Change from previous stage
Motion-Aware TCN	Temporal skeleton and global-motion backbone	70.80%	68.88%	—
TCN + 834-D geometry	Engineered clip-level geometry fusion	75.91%	74.12%	+5.11 pp Acc.; +5.23 pp F1
TCN + geometry + pairwise correction	Two conditional class specialists	**77.37%**	**76.47%**	+1.46 pp Acc.; +2.35 pp F1

The 834-dimensional geometry branch provides the largest gain, increasing Accuracy by 5.11 percentage points and Macro-F1 by 5.23 points over the TCN backbone. The pairwise correction stage provides a further improvement of 1.46 Accuracy points and 2.35 Macro-F1 points. This supports the use of explicit geometry and targeted correction under the current small-data setting.

For the interaction model, the currently available relation-only, visual-only and fused results already provide a branch-level component comparison. Further experiments can separately remove the linear SVM, the union crop and the full-scene CLIP features to identify which parts of the fusion are beneficial.

The interaction ablation follows the construction of the final model step by step. ExtraTrees is first evaluated as the non-linear relation baseline. A linear SVM branch is then added through 0.8/0.2 soft voting, after which the auxiliary CLIP branch is introduced with a visual log-probability weight of 0.1. All three variants use exactly the same five grouped folds. The final column reports both Accuracy and Macro-F1 across the five folds.

Interaction variant	Purpose	OOF Accuracy	OOF Macro-F1	Fold mean ± standard deviation
ExtraTrees only	Non-linear tree-ensemble baseline	74.23%	72.98%	Acc. 74.37% ± 8.20%; F1 73.18% ± 8.44%
ExtraTrees + Linear SVM	Contribution of the linear branch and heterogeneous soft voting	**77.32%**	75.89%	Acc. 77.42% ± 5.78%; F1 75.83% ± 6.62%
Relation ensemble + CLIP	Contribution of the auxiliary visual branch	**77.32%**	**76.13%**	Acc. 77.42% ± 6.67%; F1 76.41% ± 6.86%

The aggregated OOF metrics measure overall predictive performance, while the mean and standard deviation across folds indicate stability across data subsets. Adding the linear SVM branch to ExtraTrees raises OOF Accuracy from 74.23% to 77.32% and Macro-F1 from 72.98% to 75.89%; it also reduces the fold-level Accuracy standard deviation from 8.20% to 5.78%. Adding CLIP leaves Accuracy unchanged and further raises Macro-F1 to 76.13%. The complete model therefore achieves the highest Macro-F1 in this incremental ablation, while matching the relation ensemble in Accuracy. A formal bias–variance decomposition is unnecessary here; the controlled ablation and fold dispersion provide more directly interpretable evidence.

6.7 Runtime and Deployment Considerations

End-to-end runtime was measured by calling predict_video(task="both") three times in sequence on the development laptop: an Intel Core Ultra 7 255H with 16 logical processors, running on the CPU only. The test clips were one 4.7-second interaction test clip and one 5.0-second dog-behaviour test clip. Each call loads all models again, so the times include model loading.

Clip	Length	Detection + keypoints	Behaviour + interaction	Total
Interaction (first call)	4.7 s	47.1 s	0.6 s	47.7 s

Dog behaviour	5.0 s	32.8 s	0.8 s	33.6 s
Interaction (repeat)	4.7 s	42.9 s	0.6 s	43.5 s

A five-second clip therefore takes about 35–50 seconds, roughly seven to ten times its own length. Almost all of this time is spent in Model One: model loading, RTMDet-m on every sampled frame and two RTMPose-m passes per frame. Both recognition ensembles together, including the CLIP visual branch, take under one second.

This is acceptable for the intended use, in which clips recorded during the day are reviewed later rather than watched live. Deployment uses the following settings:

- **Sequential jobs.** The back end analyses one clip at a time, and later uploads wait in a queue. Running several CPU-heavy jobs in parallel would slow every job down.
- **Bounded memory.** Long recordings are read lazily in four-second windows, so memory use does not grow with video length. Uploads are limited to 500 MB.
- **Model sizes.** The ONNX pose models are about 54 MB each (dog and human), the RTMDet-m checkpoint is 224 MB, the dog-behaviour ensemble is 4.4 MB and the interaction ensemble is 21 MB. The CLIP ViT-B/32 encoder is loaded from a local snapshot if one exists and is otherwise downloaded once from Hugging Face.
- **Device.** Inference runs on the CPU by default. The PET_DEVICE variable selects another device.

The main ways to speed the system up are to keep the models loaded between jobs, to run detection on fewer frames and interpolate the boxes in between, and to export RTMDet to ONNX as well. None of these were needed for the prototype.

7. Findings and Discussion

7.1 Key Findings

The five-fold results support three main findings. First, preserving both local joint motion and global detection-box displacement is important for dog behaviour recognition. Second, explicit human-dog relation features are more reliable than visual appearance alone for the current interaction dataset. Third, frozen CLIP features can provide complementary object and contact context, but their contribution is smaller and should remain auxiliary under the current data scale.

7.2 Error Analysis

The main dog-behaviour errors occur between eating/drinking and standing or lying/sleeping. In several clips, the dog keeps its head low while the mouth, food or water container is occluded. The skeleton then describes head and neck posture but cannot confirm whether ingestion occurs. Future versions should add mouth-region visual information and difficult negative clips in which a dog lowers its head to sniff without eating.

For interaction recognition, petting and hand feeding both contain a hand approaching the dog's head or trunk, while playing together can also contain brief physical contact. Contact duration and the approach–hold–withdraw pattern reduce this ambiguity, but the present dataset does not cover enough variations in stroking, embracing, feeding style and viewpoint.

7.3 Impact of Detection and Tracking Quality

Both recognition tasks depend on the quality of upstream detection and pose estimation. Missing paws can change support-pattern features, inaccurate head keypoints can affect eating cues, and unstable wrist or

nose positions can strongly alter interaction distances. Confidence-weighted pooling, temporal interpolation and sequence smoothing reduce short errors, but long occlusions cannot be reconstructed reliably. In the current one-person, one-dog scope, identity tracking is not required for resolving multiple subjects; nevertheless, consistent detections across the 32-frame window remain necessary.

7.4 Data Quality, Size and Leakage

The dataset was organised by source video and subject identity rather than by randomly splitting individual frames. This prevents near-duplicate clips from the same video from appearing across training and evaluation partitions. External APT skeleton sequences are restricted to the training folds. Despite these controls, the number of identities and clips remains limited, particularly for subtle or visually dependent categories. Consequently, five-fold OOF metrics are treated as the principal results, while the small test sets are used only to demonstrate feasibility.

7.5 Limitations

The current system assumes one principal person and one principal dog and does not resolve identity association in multi-person or multi-dog scenes. The pose representation is two-dimensional and is sensitive to viewpoint, self-occlusion and truncation. The visual branch uses a frozen general-purpose CLIP encoder rather than a model adapted specifically to indoor human-dog interaction. Finally, the predicted labels describe observable actions only and should not be interpreted as diagnoses of animal health, emotion or safety.

7.6 Future Work

Future work should collect more identity-independent videos for the weak and confusable classes, especially eating/drinking, petting and hand feeding. A larger locked evaluation set would provide a more stable final estimate. Model development could also investigate mouth-centred visual features, more explicit contact-duration modelling, three-dimensional pose cues, and multi-subject association. Once sufficient data are available, the TCN, ST-GCN and Transformer alternatives should be rerun under the same grouped protocol for a complete controlled comparison.

8. Conclusion

This project built a working prototype that turns short indoor home-camera clips into dog-behaviour and human–dog interaction labels with confidence scores and a timeline. It runs entirely on a laptop CPU, from upload to the mobile results page.

The work produced three main results:

- **Pose estimation.** The official RTMPose-m AP-10K model (P0) already performs close to its own benchmark on indoor dogs. Fine-tuning on APT-36K raised accuracy on APT-36K by about four AP points but did not improve the indoor frames, so P0 was kept. The bidirectional, score-aware tracking rule was the more effective intervention: it recovered most low-scoring frames of lying and partly hidden dogs that a fixed threshold would have discarded.
- **Dog behaviour.** A motion-aware TCN combined with explicit geometry features and two pairwise specialists reaches 77.37% OOF accuracy and 76.47% Macro-F1 on five classes. The engineered geometry added about five points over the TCN alone.
- **Human–dog interaction.** Relation features between the two skeletons, classified by an ExtraTrees/linear-SVM ensemble, reach 77.32% OOF accuracy. A frozen CLIP branch with a small fusion weight raised Macro-F1 to 76.13%.

Across the three models, explicit domain features (detection-box motion, head-lowering duration and hand-to-nose distance) were more valuable than larger learned models under the available data. All evaluation used splits grouped by dog, person and source video.

The remaining errors are concentrated in eating/drinking versus standing or lying, and in hand feeding versus petting. Both are cases where the skeleton cannot show whether food or contact is actually present. The system assumes one principal dog and one principal person, uses two-dimensional pose and was evaluated on small held-out test sets. More identity-independent data for the confusable classes, mouth-centred visual cues and multi-subject tracking are the most direct next steps (Section 7.6).

References

1. Bai, S., Kolter, J. Z., and Koltun, V. (2018). *An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling*. arXiv:1803.01271.
2. Geurts, P., Ernst, D., and Wehenkel, L. (2006). "Extremely Randomized Trees." *Machine Learning*, 63, 3–42.
3. Radford, A., et al. (2021). "Learning Transferable Visual Models From Natural Language Supervision." *Proceedings of the 38th International Conference on Machine Learning*.
4. Yan, S., Xiong, Y., and Lin, D. (2018). "Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition." *Proceedings of the AAAI Conference on Artificial Intelligence*.
5. Lyu, C., Zhang, W., Huang, H., et al. (2022). *RTMDet: An Empirical Study of Designing Real-Time Object Detectors*. arXiv:2212.07784.
6. Jiang, T., Lu, P., Zhang, L., et al. (2023). *RTMPose: Real-Time Multi-Person Pose Estimation based on MMPose*. arXiv:2303.07399.
7. Yu, H., Xu, Y., Zhang, J., Zhao, W., Guan, Z., and Tao, D. (2021). "AP-10K: A Benchmark for Animal Pose Estimation in the Wild." *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*.
8. Yang, Y., Yang, J., Xu, Y., Zhang, J., Lan, L., and Tao, D. (2022). "APT-36K: A Large-scale Benchmark for Animal Pose Estimation and Tracking." *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*.
9. Lin, T.-Y., Maire, M., Belongie, S., et al. (2014). "Microsoft COCO: Common Objects in Context." *European Conference on Computer Vision*.
10. MMPose Contributors (2020). *OpenMMLab Pose Estimation Toolbox and Benchmark*. https://github.com/open-mmlab/mmpose.
11. Li, Y., Yang, S., Liu, P., et al. (2022). "SimCC: A Simple Coordinate Classification Perspective for Human Pose Estimation." *European Conference on Computer Vision*.
12. Grand View Research (n.d.). *Pet Monitoring Camera Market Size, Share & Trends Analysis Report*. Grand View Research.

Appendix

A. Installation and User Guide

Requirements. The system needs Conda with Python 3.11, Node.js for the front end, and FFmpeg on the path. Python 3.11 is required because the released scikit-learn models were saved with scikit-learn 1.9.1, and pickled models load reliably only with the version that saved them.

Back-end and inference environment. Run from pet_recognition_system/:

```
conda create -n PRS-project-311 python=3.11 -y
conda activate PRS-project-311
pip install -r requirements.txt          # ONNX inference and keypoint evaluation
pip install -r requirements-training.txt   # PyTorch, scikit-learn 1.9.1, joblib
pip install -r requirements-app.txt        # FastAPI, Uvicorn, pytest
pip install -e .
```

Data root. Set PET_DATA_ROOT to the local data folder. It must contain the three pose-pipeline weights:

```
<PET_DATA_ROOT>/models/
├── rtmdet-m_coco/rtmdet_m_8xb32-300e_coco_20220719_112220-229f527c.pth
├── rtmpose-m_coco_human/end2end.onnx
└── rtmpose-m_ap10k/end2end.onnx
```

The frozen CLIP model openai/clip-vit-base-patch32 is read from <PET_DATA_ROOT>/models/clip-vit-base-patch32/ if that folder exists. Otherwise it is downloaded on the first run.

Running the application. Start the back end and the front end in two terminals:

```
cd pet_recognition_system
PET_DATA_ROOT=<data-root> uvicorn app.server:app --port 8000
cd frontend
npm install
npm run dev        # opens the app; /api is proxied to localhost:8000
```

The development server also prints a LAN address, which a phone on the same Wi-Fi network can open.

npm run dev:mock runs the same pages on sample data without the back end, and npm run build produces a static build in frontend/dist/.

Using the application.

1. Open **Upload**, choose a clip, check the recording time and camera, and submit.
2. The **Analyze** page shows the four analysis stages. When the job finishes, open the result.
3. **Result** shows the dog's behaviour, the interaction and the timeline of the clip.
4. **Story** collects the clips of one day, and **Week** summarises the behaviour minutes of the week.

Command-line prediction. One video can be analysed without the application:

```
python tools/predict_video.py input.mp4 --task both --data-root <PET_DATA_ROOT>
```

--task dog_behavior or --task interaction runs only one model, and --export-clips cuts each merged timeline segment into its own video.

Tests. Run python -m pytest -q in pet_recognition_system/ and npm test in frontend/.

B. Model Training and Reproduction Commands

All commands are run from pet_recognition_system/ with the training requirements installed.

For dog-behaviour training, <dog-skeleton-dir> must contain the files from both Hugging Face folders data/skeletons_generated/ and data/skeletons_generated_APT/, with each APT file renamed to dog_<clip_id>.npz. The full option lists are in MODEL_TRAINING.md.

Model One: dog pose.

```
python tools/eval_pose.py P0=$PET_DATA_ROOT/models/rtmpose-m_ap10k/end2end.onnx P1=checkpoints/p1_apt36k.onnx
python tools/detect_boxes.py                # RTMDet boxes for the real-pipeline setting
python tools/eval_pose.py P0=$PET_DATA_ROOT/models/rtmpose-m_ap10k/end2end.onnx --boxes outputs/eval_boxes_rtmdet.json
```

```
python tools/prepare_apt36k_split.py              # APT-36K train/val split by video
python tools/train_pose.py configs/rtmpose/rtmpose-m_apt36k-dog_finetune.py
python tools/export_rtmpose_onnx.py configs/rtmpose/rtmpose-m_apt36k-dog_finetune.py \
   work_dirs/rtmpose-m_apt36k-dog_finetune/best_coco_AP_epoch_N.pth checkpoints/p1_apt36k.onnx
```

Skeleton extraction.

```
python tools/extract_skeletons.py CLIP.mp4 [CLIP.mp4 ...] --out-dir outputs/skeletons [--video]
```

Model Two: dog behaviour.

```
python tools/cross_validate_dog_behavior.py \
  --manifest outputs/best_models/manifests/dog_behavior_manifest_5class_v4.csv \
  --skeleton-dir <dog-skeleton-dir> --model motion_aux_tcn --folds 5 --epochs 80 \
  --patience 15 --target-frames 32 --seed-base 28 --output-dir work_dirs/dog_behavior_5class_v4_cv
python tools/cross_validate_dog_5class_fusion.py \
  --cv-dir work_dirs/dog_behavior_5class_v4_cv --skeleton-dir <dog-skeleton-dir> \
  --feature-version head_duration_v1 --folds 5 --seed 28 \
  --output-dir work_dirs/dog_behavior_5class_v5_head_duration_fusion
python tools/evaluate_dog_5class_ensemble.py \
  --manifest outputs/best_models/manifests/dog_behavior_manifest_5class_v4.csv \
  --skeleton-dir <dog-skeleton-dir> --cv-dir work_dirs/dog_behavior_5class_v4_cv \
  --fusion-dir work_dirs/dog_behavior_5class_v5_head_duration_fusion \
  --feature-version head_duration_v1 --output-dir work_dirs/dog_behavior_retrained_test
```

Model Three: human–dog interaction.

```
python tools/cross_validate_interaction_relation.py \
  --manifest outputs/best_models/manifests/interaction_manifest_mixed_v2.csv \
  --skeleton-dir <data>/skeletons_generated --folds 5 --augmentations 0 --seed 28 \
  --classifier soft_vote --max-features sqrt --min-samples-leaf 1 \
  --hand-feeding-weight 1.5 --svm-c 0.005 --svm-weight 0.2 \
  --output-dir work_dirs/interaction_relation_mixed_v2_cv5_noaug
python tools/train_final_interaction_ensemble.py \
  --manifest outputs/best_models/manifests/interaction_manifest_mixed_v2.csv \
  --skeleton-dir <data>/skeletons_generated --output-dir work_dirs/interaction_relation_mixed_v2_final \
  --models 5 --seed 28 --classifier soft_vote --max-features sqrt --min-samples-leaf 1 \
  --hand-feeding-weight 1.5 --svm-c 0.005 --svm-weight 0.2
python tools/extract_interaction_clip_features.py \
  --manifest outputs/best_models/manifests/interaction_manifest_mixed_v2.csv \
  --video-dir <data>/interaction --skeleton-dir <data>/skeletons_generated \
  --output-dir outputs/interaction_clip_features_clip_v2 --frames 8
python tools/cross_validate_interaction_rgb_fusion.py \
  --manifest outputs/best_models/manifests/interaction_manifest_mixed_v2.csv \
  --skeleton-dir <data>/skeletons_generated --feature-dir outputs/interaction_clip_features_clip_v2 \
  --base-oof-dir work_dirs/interaction_relation_mixed_v2_cv5_noaug \
  --base-final-dir work_dirs/interaction_relation_mixed_v2_final \
  --output-dir work_dirs/interaction_multimodal_release --deployment-alpha 0.1
python tools/evaluate_interaction_multimodal.py \
  --model-root checkpoints/best_models/interaction \
  --manifest outputs/best_models/manifests/interaction_manifest_mixed_v2.csv \
  --skeleton-dir <data>/skeletons_generated --feature-dir outputs/interaction_clip_features_clip_v2 \
  --split test --output-dir work_dirs/interaction_multimodal_release_check
```

Reproducing the released results. The released ensembles can be evaluated directly by passing --cv-dir checkpoints/best_models/dog_behavior/backbone and --fusion-dir checkpoints/best_models/dog_behavior/fusion to evaluate_dog_5class_ensemble.py, and --model-root checkpoints/best_models/interaction to evaluate_interaction_multimodal.py. Downloaded files can be verified against checkpoints/best_models/SHA256SUMS.txt.

C. Dataset and Model File Listing

Public dataset. The clips and generated skeletons are published on Hugging Face as melodyyyyy123/PRS-Group28-data2:

- data/skeletons_generated/ holds the project skeletons;
- data/skeletons_generated_APT/ holds the 31 APT training sequences.

Each skeleton file <clip_id>.npz contains:

Key	Shape	Content
skeletons	(2, T, 17, 3)	Stream 0 person (COCO-17), stream 1 dog (AP-10K-17); x, y in pixels and confidence
boxes	(2, T, 5)	Person and dog box x_1, y_1, x_2, y_2 and detector score
fps	scalar	15
image_size	(2,)	Width and height in pixels

Local data root (PET_DATA_ROOT).

Path	Contents
raw/dog_behavior/	Behaviour clips (<clip_id>.mp4) and dog_behavior_clips.csv
raw/interaction/	Interaction clips and clips.csv
raw/skeletons_generated/, raw/skeletons_generated_APT/	Extracted skeleton files
APT-dog/, apt36k_dog_train.json, apt36k_dog_val.json	APT-36K dog frames and the video-level split
评估/	The 36 hand-labelled LabelMe evaluation frames (Section 2.4)
models/	RTMDet-m COCO, RTMPose-m AP-10K (dog) and RTMPose-m COCO (human) weights

Manifests and released models (in the repository).

Path	Contents	Size
outputs/best_models/manifests/dog_behavior_manifest_5class_v4.csv	Behaviour clips, labels, splits and folds	–
outputs/best_models/manifests/interaction_manifest_mixed_v2.csv	Interaction clips, labels and splits	–
checkpoints/best_models/dog_behavior/	backbone/fold_0..4/best.pt, fusion/*.joblib, config.json	4.4 MB
checkpoints/best_models/interaction/	checkpoints/fold_1..5.joblib, rgb/fold_1..5.joblib, config.json	21 MB
checkpoints/p1_apt36k_epoch19.onnx	P1 dog pose model (not used in the final system)	54.5 MB
checkpoints/best_models/MODEL_CARD.md,	Model card and checksums	–

SHA256SUMS.txt		

D. Additional Results

Dog behaviour, per-class results. The values below are taken from the five-fold OOF evaluation on 137 clips and the 33-clip test set.

Class	OOF precision	OOF recall	OOF F1	OOF support	Test precision	Test recall	Test F1	Test support
Walking / Running	0.947	0.818	0.878	22	1.000	0.833	0.909	6
Standing	0.586	0.607	0.596	28	0.700	0.875	0.778	8
Sitting	0.818	0.900	0.857	30	1.000	0.833	0.909	6
Lying / Sleeping	0.825	0.825	0.825	40	0.750	0.857	0.800	7
Eating / Drinking	0.688	0.647	0.667	17	0.600	0.500	0.545	6

Dog behaviour, OOF confusion matrix (counts). Rows are the actual class and columns the predicted class, in the order walking/running, standing, sitting, lying/sleeping and eating/drinking.

Actual Predicted	Walk	Stand	Sit	Lie	Eat
Walking / Running	18	3	0	0	1
Standing	1	17	4	3	3
Sitting	0	2	27	1	0
Lying / Sleeping	0	4	2	33	1
Eating / Drinking	0	3	0	3	11

Interaction, per-class results. The values below are taken from the five-fold OOF evaluation on 97 clips and the 24-clip test set.

Class	OOF precision	OOF recall	OOF F1	OOF support	Test precision	Test recall	Test F1	Test support
No interaction	0.929	0.963	0.945	27	1.000	0.833	0.909	6
Hand feeding	0.632	0.522	0.571	23	0.833	0.833	0.833	6
Petting	0.720	0.720	0.720	25	0.667	0.667	0.667	6
Playing together	0.760	0.864	0.809	22	0.857	1.000	0.923	6

Interaction, OOF confusion matrix (counts).

Actual Predicted	None	Feeding	Petting	Playing
No interaction	26	0	1	0
Hand feeding	2	12	6	3

Petting	0	4	18	3
Playing together	0	3	0	19

Dog pose, PCK@0.1 per keypoint. The values below are measured on the 36 indoor frames, with RTMDet boxes.

Keypoint	Labelled	P0	P1	Keypoint	Labelled	P0	P1
Left eye	33	0.970	0.970	Right elbow	32	0.875	0.906
Right eye	33	0.879	0.909	Right front paw	31	0.871	0.871
Nose	34	0.853	0.882	Left hip	35	0.714	0.714
Neck	36	0.861	0.889	Left knee	30	0.900	0.833
Root of tail	34	0.912	0.853	Left back paw	28	0.893	0.857
Left shoulder	35	0.829	0.800	Right hip	33	0.788	0.758
Left elbow	33	0.909	0.970	Right knee	26	0.923	0.885
Left front paw	33	0.909	0.939	Right back paw	26	0.846	0.846
Right shoulder	35	0.686	0.771				